

DESODBC v1.1: Software design description

The ODBC (Open Database Connectivity) standard, developed by Microsoft and Simba, describes an API from the SQL standard to access data sources, mainly relational databases. The majority of the database management systems (and even other systems like MS Excel) provide their own ODBC Driver for accessing their own data sources. This work presents the development of a new ODBC driver named DESODBC for the database management system DES (Datalog Educational System) developed by Fernando Sáenz-Pérez, which supports SQL among other query languages. In the development we will start from the open-source ODBC Driver for MySQL (MyODBC), adapting it properly to the characteristics of DES. Finally, the driver offers versions for Windows, GNU/Linux and macOS, which are the platforms in which DES is supported.

This work is a direct derivate from the undergraduate thesis of Sergio Garcia-Jimenez *Conector ODBC para DES*, tutored by Fernando Saenz-Perez, URI: <https://hdl.handle.net/20.500.14352/123858>. It retains the same copyright and related rights as the original thesis.

Sergio Miguel García Jiménez

Keywords

ODBC, DES, SQL, Datalog, MyODBC.

Table of contents

1. Introduction	1
Introduction	1
1.1. Preliminaries and motivation	1
1.1.1. What is an ODBC?	1
1.1.2. Datalog Educational System (DES)	1
1.1.3. Proposed problem	2
1.2. Objectives	2
1.3. Work plan	4
1.4. Final product	5
1.5. Organization of the document	5
2. State of the question	7
2.1. Preliminaries: DES and ODBC	7
2.2. On the development of an ODBC connector, and how it would be in DES	8
2.2.1. Some preliminar aspects specific from DES	8
2.2.2. Why we started from the MySQL driver	9
2.2.3. DESODBC schema	10
2.2.4. Concurrency policies	11
3. Work description	13
3.1. Import and compilation of MyODBC	13
3.2. Preliminaries: data sources	14
3.3. Development of the installer	15
3.4. Development of the setup library	15
3.5. Development of the ODBC driver	18
3.5.1. Main classes and getter/setter functions	20
3.5.2. Environment and connection functions	23
3.5.3. Statement handling functions	43
3.5.4. Result handling functions	45
3.5.5. Catalog functions (metadata)	50

3.5.6. Error management system	61
3.5.7. ANSI and Unicode versions of the driver	63
3.6. <i>Testing</i>	65
3.7. Documentation	66
3.8. On development and debugging: software used	67
3.8.1. Independent of the system	67
3.8.2. Windows	67
3.8.3. Unix	71
4. Conclusions and future work	73
Conclusions and future work	73
4.1. Conclusions	73
4.2. Future work	73
Bibliography	75

Introduction

1.1. Preliminaries and motivation

This work consists of an implementation of an ODBC Driver for the DBMS (Data Base Management System) Datalog Educational System (DES).

1.1.1. What is an ODBC?

ODBC (Open Database Connectivity) is «*a standard application programming interface (API) for accessing data in both relational and nonrelational database management systems (DBMSs)*» [1, pp. 3-4], of last version 3.8 (2009) [2]. This standard was developed in the early 90s by Microsoft and Simba Technologies, and solved a previously impracticable problem: being able to access multiple databases through the SQL standard from a single application, without having to program specific code in there for these connections [1, p. 3].

Thus, a DBMS could have its own ODBC driver that supported ODBC API function calls for communication with one of the DBMS's data sources.

Therefore, an application wishing to connect to such data source would only need to communicate with an ODBC Driver Manager (for instance, in Windows, `System32/odbc32.dll`) that acted as an intermediary between the application and the connector. In this way, communications between the application and the desired data source would take place without needing any extra programming; that is, it would be agnostic to the driver's implementation and the nature of the data source or its associated DBMS.

1.1.2. Datalog Educational System (DES)

Datalog Educational System (DES) is a deductive database management system which is open-source, free, cross-platform, and published originally in 2004 by Fernando Sáenz-Pérez, Associate Professor / *Profesor Titular* of Universidad Complutense de Madrid [3]. DES, among many of its characteristics, supports queries in the languages Datalog, SQL, Relational Algebra, Tuple Relational Calculus and Domain Relational Calculus [4].

The developer, who has also been responsible for its maintenance until today, had as his main objective the use of DES in the educational field for students learning databases, aiming to teach intuitively and interactively the concepts behind a deductive database, as well as teaching the supported languages for queries that we have listed [5, p. 10]. However, its use spread to universities all around the world, having as a collateral effect even its use in research in different fields [6].

The author of this work was one of these students who were introduced to this system, with the developer himself as his professor in the course *Bases de Datos* (Databases) in third year of the *doble grado en Ingeniería Informática y Matemáticas* ("double degree in Computer Science and Mathematics"); this professor is also the tutor of this work.

1.1.3. Proposed problem

The director of this work described the problem in the following lines¹:

«ODBC (Open Database Connectivity) is a standard API for accessing data sources, mainly relational databases, developed by Microsoft and Simba. Currently there are ODBC drivers for the majority of databases (MySQL, PostgreSQL, DB2...) and even to other sources such as Excel and CSV files. In this project a new ODBC driver will be developed for the deductive database system DES (<http://des.sourceforge.net>), that supports various query languages on relational data structures. The usual programming language used for developing these drivers is C, and one can start from the Microsoft docs and already done open-source implementations such as those for PostgreSQL and MySQL. Given that DES does not support concurrent queries, it will be necessary to implement its management in the proper driver. Depending on the number of students, connectors can be developed not only for Windows, but also for Linux and macOS. » [7]

Indeed, even though DES is able to connect to a ODBC Driver for communicating with other DBMSs, DES lacks its own ODBC driver. That is, there is no driver through which a ODBC Driver Manager could communicate with DES, so that external applications could connect to specific databases of this DBMS.

1.2. Objectives

Once we have present the proposed problem, the objective of this work is to develop an ODBC Driver, understanding that its fundamental characteristics will be: (1) supporting calls from the ODBC Driver Manager, which demands some information; (2) access to the DES databases through its own already implemented API (Textual API or TAPI, as we will see), and (3) return the requested information to the ODBC Driver Manager. For this, we will start from the source code of MyODBC, the ODBC Driver of MySQL.

¹I have translated it into English from the Spanish original.

We will implement a ODBC Driver for the last version of ODBC (3.8) for the last version for DES (V6.7). We will propose as the concrete minimal objective offering a ODBC Driver for DES in all the operating systems that DES supports: Windows, GNU/Linux and macOS.

Given that we start from the already implemented MyODBC driver for the DBMS MySQL, we will also propose as optional objectives implement all the functionalities that its suite offers, namely:

- Installer executable capable of:
 - Registering the ODBC driver in the system.
 - Adding and updating data sources.
- GUI (Graphical User Interface) setup in Windows for the parameters of a data source associated to the ODBC driver².
- The ODBC driver itself, with versions supporting:
 - Operating systems Windows, GNU/Linux with unixODBC, macOS with unixODBC and iODBC³.
 - ANSI and Unicode formatting⁴.

In synthesis, we will satisfy the main objective but we will also strive to achieve these last optional objectives. With this, we will accomplish more than enough the director's proposed objectives in the presentation of the problem (section 1.1.3).

With the implementation of this software, we will have revised all the software concepts that are taught in the current study guidelines: we deal with object-oriented programming, databases, data structures, algorithms, concurrency policies, IPC mechanisms, GUI designing (and its respective criteria of proper interactive systems designing), correctness, internal aspects of operating systems, use of software development tools, etc.

²MyODBC in its source code provides means for compiling GUI setup libraries for GNU/Linux. These libraries are not distributed in the official versions of MyODBC, so as a result we will not include providing setup libraries for GNU/Linux as a requisite. However, we have worked on these extra-official libraries and we had included them in the final work with some considerations (see section 3.4).

³In reality, the MyODBC driver supports other *Unix-like* systems such as Oracle Solaris. DES can be used in this last operating system if both DES and its dependencies SWI-Prolog/SICStus are compiled from their respective sources. DES does not present an official version for Oracle Solaris: consequently, we then restrict ourselves to the systems in which DES offers support: Windows, GNU/Linux, macOS.

However, we believe that, excepting any minor issues that may need to be resolved, this driver can be compiled and work in Oracle Solaris. The reason why is that, as we will see, our implementation is practically agnostic to the Unix-like systems in which it is destined, with some minor exceptions regarding differences relative to POSIX functions.

⁴In the section 3.5.7 we will study what we are referring to.

1.3. Work plan

1. Research phase (*July 2024 - August 2024, January 2025 - March 2025*). The questions we will need to answer are:

- What is ODBC.
- What is a ODBC Driver.
- How is the usage flow of an ODBC Driver and the agents implied in it.
- Which DES functionalities we will need to use.
- What functionalities MyODBC has.
- What MyODBC means by ANSI and Unicode.
- How an ODBC connection works low-level in Windows and *Unix-like* systems.
- What functionalities of Windows API and POSIX can we use.

Part of these questions will be answered and researched throughout our development phase while we find unpredictable challenges. This is why the research and development phases are broad, general tasks that intertwine and overlap in time.

2. Development phase (*July 2024 - August 2024, January 2025 - March 2025*). The working flux we have considered best is the following:

- a) Importing and compiling the MyODBC source code in Windows.
- b) Parallel process of adapting the driver to the DES needs, and removal of MySQL-related external libraries that we do not need.
- c) Adaptation to the *Unix-like* systems GNU/Linux and macOS.

3. Testing and debugging phase (*April 2025 - May 2025*).

- Testing with typical use cases as well as «pathological» ones so as to debug possible concept errors, bugs, and security flaws.
- Debugging and refinement of the driver beyond that of the development phase.

4. Writing of the final report and documenting the source code (*April 2025 - May 2025*).

The figure 1.1 shows the Gantt chart that portrays the flow of the concrete and broken-down tasks.

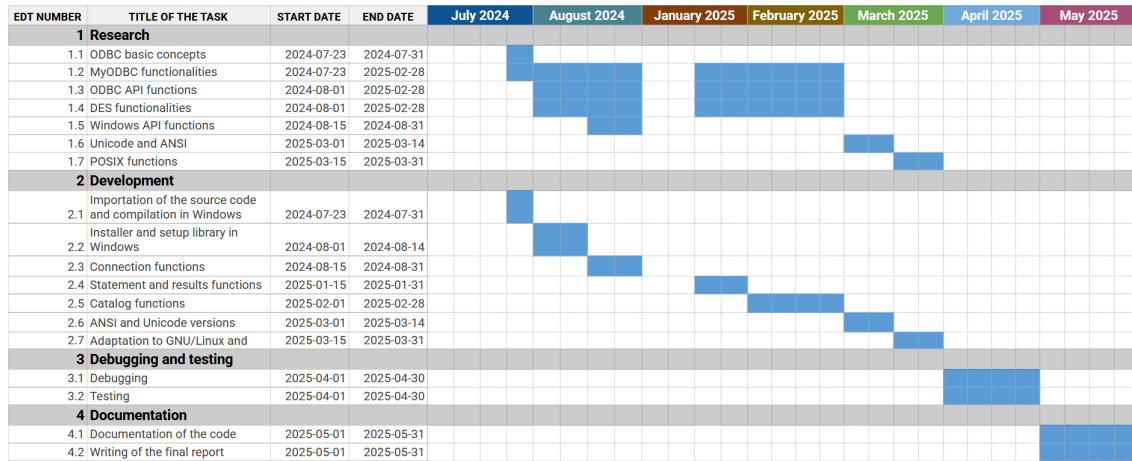


Figure 1.1: Gantt chart associated with our work plan.

1.4. Final product

The author of this work would end up doing not only the Windows implementation but also the GNU/Linux and macOS ones. In the director's proposed objectives it is explained that the scenery of exporting to different operating systems would apply in the case of multiple students. We explain at this point that, as we will see, the code of the functionalities themselves of the driver is directly exportable for all the compilers and operating systems; the only thing that we will have to take into account for each system, is the aspect of connecting to a DES process and the implied Inter-Process Communications (IPCs) implied, as well as some aspects regarding the assistant tools (setup and installer, as we will see). This is the reason why the author of this work considered that the extra work to do was not enough for requiring a team.

Our final product is an ODBC Driver called DESODBC and its GitHub repository where it is stored can be visited at the following link: <https://github.com/segar21/des-connector-odbc>.

1.5. Organization of the document

This document describes in the Chapter 3 the development of the research and implementation phases in parallel for didactic and exhibition purposes. Conclusions in English will be described in Chapter 4.

State of the question

2.1. Preliminaries: DES and ODBC

An illustrative schema showing the ODBC architecture can be seen in Figure 2.1.

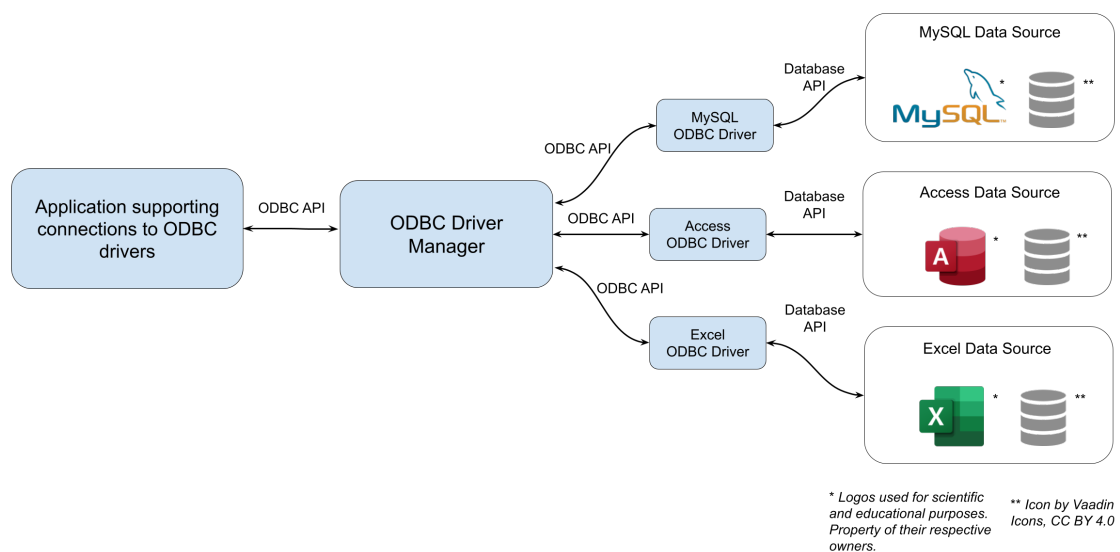


Figure 2.1: Schema example of the ODBC model.

Developing a more detailed exposition of the motivation, DES can establish connections to external databases through the ODBC standard: this is, DES can operate at the application level as shown in Figure 2.1, and can connect to any data source, such as databases with DBMS supporting ODBC connections (MySQL, MS Access, Oracle, ...) [5, p. 13]. Connections are made using the `/open_db` command (see Figure 2.2), with the possibility of closing them using the `/close_db` command¹.

As previously mentioned, DES lacks its own ODBC driver, and its implementation is the main objective of this work.

¹More information on ODBC support is found in [5, section 5.1].

```
DES> /open_db mysql
DES> select * from edge
answer(a:integer(4),b:integer(4)) ->
{
    answer(1,2),
    answer(2,3),
    answer(3,4)
}
Info: 3 tuples computed.
```

Figure 2.2: Example of connection to the MySQL ODBC connector [5, section 5.1.8].

2.2. On the development of an ODBC connector, and how it would be in DES

2.2.1. Some preliminar aspects specific from DES

Firstly, we point out that we do not need to worry about the *Database API* layer presented in Figure 2.1, as DES implements an API through which an external application can communicate with its databases, presenting the required information in an easily analyzable format. This API was named by the author of DES as TAPI (*Textual API*), and is activated by adding a prefix `/tapi` to the queries we send to DES [5, sección 5.18].

Let's delve further into the operation of TAPI, as this API will form the basis for input and output between our connector and DES.

Our connector will be able to read and write about a DES process using the standard input, output, and error streams. However, analyzing from a program an output that is written in a human-readable format is complicated. Sending the query with the `/tapi` prefix allows obtaining an output that is much easier to extract information from algorithms written in our connector. In Figure 2.3 we illustrate the difference between obtaining an output in human-readable format and in TAPI format.

As can be seen in said figure, the TAPI format presents an output that is easier to analyze for a computer program. After a `SELECT` operation, the output starts with the keyword `answer`. The character `$` is the delimiter between the different attribute lists (in the example, in order: column names and types, first column, second column) and `$eot` indicates the end of the output. There are different TAPI output formats depending on the command, and the complete specification of TAPI can be found in the DES manual [5, section 5.18].

<pre>DES> select * from people answer(people.id:int, people.name:string) -> { answer(23,'anderson'), answer(57,'nolan') } Info: 2 tuples computed.</pre>	<pre>DES> /tapi select * from people answer people.id int people.name string \$ 23 'anderson' \$ 57 'nolan' \$eot</pre>
---	--

Figure 2.3: Comparison between standard and TAPI queries.

Lastly, we must note that an ODBC connector for DES will not be able to exploit all the features of this system. This is because DES is not just a relational DBMS, but rather a highly versatile deductive database management system, which in particular offers aspects of relational databases as a possibility (DES implements the SQL standard). Therefore, our connector will not be able to handle all aspects related to *Datalog* natively, as it falls outside the scope of ODBC. Our connector will provide the ODBC API functions, so only functions related to the SQL standard will be offered.

However, with our connector, it is possible to work with all aspects of DES to some extent, as there is the option to send direct commands to the DES console (using ODBC API functions of the type *execute*) and view the corresponding output, which enables the exploitation of all DES features from the application connected to the ODBC connector.

2.2.2. Why we started from the MySQL driver

Developing an ODBC connector for a DBMS from scratch is a highly complex task that requires advanced knowledge of various aspects to be considered in the connector (data types, character encoding, concurrency, compatibilities, the full range of errors to be displayed to the user, etc.). Thus, as the project manager suggested, we consider it intelligent to study an existing ODBC connector and progressively adapt it to the particularities of DES.

For this purpose, we have chosen the MyODBC connector (MySQL Connector/ODBC), since it is possible to study, modify, and distribute it being an open-source connector (*GPLv2*) and MySQL being one of the most popular DBMSs in the world, with the advantages this entails for solving possible incidents in our development.

First, let's see the characteristics of DES and how our connector should behave in accordance with these.

2.2.3. DESODBC schema

When an application such as MySQL, Access, or any other wants to connect to an ODBC, it first loads a copy of the ODBC Driver Manager (ODBC Driver Manager on Windows; unixODBC on Unix-based systems such as GNU/Linux, macOS, and Oracle Solaris; and also iODBC for macOS) into its memory space ².

When the application wishes to connect to an ODBC connector, it allocates space for a connection object (which we will see later) and requests the ODBC Driver Manager that connects to the specified data source. The ODBC Driver Manager loads into the application's memory space a copy of the library that corresponds to the connector.

Now the question arises of how to design the architecture. We have many possibilities depending on the degree of resource sharing and visibility that the different connections should have: from having a global instance of DES for all connections, to the other extreme where each connection opens a DES process.

However, our architecture will be determined by the following problem: there is a conflict when two distinct applications open DES processes from the same working directory. In these circumstances, there is a conflict with the state file, which refers to the state of the local database of DES. For example, when the command `/save_state` is called in DES, the state is saved in a `des.sds` file in the working directory from which DES is initialized; and in general, by default, the commands received by DES are executed from paths relative to the working directory. This is why different applications that aim to work in the same working directory must necessarily connect to a single shared DES process. We chose to open a DES process for each connection launched from a new working directory^{3 4}.

We illustrate the resulting schema in Figure 2.4, which represents the case for Windows (analogous in fundamentals for other platforms). Unix-based systems.

Thus, as we can see in Figure 2.4, MySQL establishes two separate connections to DES from two different working directories; therefore, two distinct processes are

²In Windows, when applications call the corresponding `.dll` for our connector, a copy is loaded (Windows loads instances of the libraries in the address space of the application that refers to these libraries [8]). The same will happen with all DLL libraries, in particular also with our connector.

The equivalents of DLLs for GNU/Linux and MacOS (`.so`, `.dylib`) present analogously the same loading behavior with copy.

³Something similar is done in DESweb, an application in the DES ecosystem that provides access to DES in the web environment <https://desweb.fdi.ucm.es/>. The underlying server runs a single DES process for each working directory, which refers to the directory generated for a client who connects.

⁴Even if applications refer to different DES executables, this still applies. Generally, DES would be installed in a single directory, but it could also be installed in multiple different paths due to user requirements: in such a case, there would still be a conflict if we consider different DES executables in the same working directory, as the different processes would write to the same working directory. Thus, when application B attempts to connect to a workspace already in use by another connection A, if the pre-existing global DES process associated with that workspace refers to an executable different from the one required by B, B will not be able to connect (different executables inevitably involve different processes). B will be notified that it must refer to the path of the executable used in A's connection.

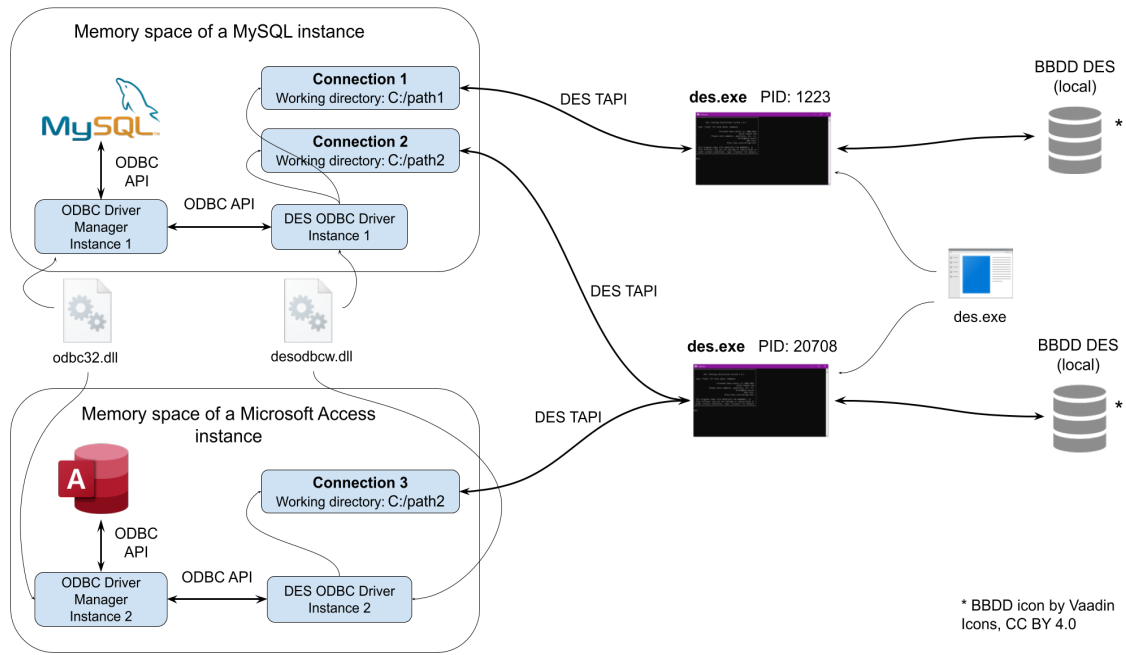


Figure 2.4: Illustrative schema of DESODBC's behavior.

opened. However, Access connects to DES from a working directory that turns out to share another connection made by MySQL (`C:/path2`) and therefore, proceeds to share said already open process with MySQL.

2.2.4. Concurrency policies

The fact that DES is an embedded or local DBMS (i.e., it is directly integrated into the application and does not require network processing, such as SQLite) facilitates the development of concurrency. Moreover, DES does not provide transactions, levels of isolation and other security mechanisms since it is not a complete deductive DBMS [5, p. 10].

However, the fact that two distinct connections may refer to the same DES process requires us to implement some concurrency policies. Regarding DES itself, we do not need to consider any aspects of concurrency, as DES does not support transactions, and concurrency is handled outside of the DES processes regarding reading/writing system files (through `COMMIT`, `ROLLBACK` and other DES special commands) without any concurrency beyond that implemented by the operating system itself.

However, for connections that use the same DES process, it will be essential to implement safe and effective concurrency mechanisms to resolve what would initially seem to be a problem of concurrency *readers/writers* (*R/W*) with respect to the input/output of the console associated with a shared DES process.

The implementation of concurrency between the connector instances and this single DES process will be based on shared memory. The connector instances will access data that allows communication with the single process DES (a single series

of pipes) by scheduling them with simple, coarse-grained concurrency policies. This is because our R/W problem is simplified: each connection simply acquires a lock, which it will retain possession of for the entire duration of the query process (query sending and receiving of the result), after which it proceeds to free the lock.

This has the additional advantage that subsequent connections attempting to make new queries will not be able to read commands executed previously by others, since the reading pipeline is consumed within each information exchange process. This is why our R/W problem is not the classic R/W problem considered in computer science literature.

Our concurrency implementation involves more details than the classic R/W we have addressed in this section. We will see all the considerations we took into account later on.

Work description

In this chapter, all the work carried out is described. The order followed here does not correspond to the work plan outlined in section 1.3: for the purposes of presentation and pedagogy, we will attempt to describe, in a parallel and interconnected manner, theoretical concepts of ODBC and implementation aspects. After describing the connector's development, we include sections to detail the software used, testing, and documentation.

3.1. Import and compilation of MyODBC

The version of MyODBC used was 9.0.0, and as a first step, we imported and compiled it directly on Windows.

We do not provide a detailed description of the import and compilation steps. However, the task itself was relatively complex due to the numerous challenges we encountered when compiling such complex software. MyODBC offers documentation for compiling the source on Windows (see [9]), but we had to deal with the issue of compiling the project in Debug mode. The problem arises when external dependencies (such as MySQL Server, `mysqlclient.lib`) are compiled in Release mode. Therefore, we had to compile the dependencies in Debug mode, which was a cumbersome task as these dependencies sometimes relied on others.

This is why we invest significant effort and time to compile the MyODBC project on Windows in **Debug** mode before starting to work with it. However, once our connector is developed and all external library dependencies are removed, the project becomes self-contained. In this way, the project was exported to GNU/Linux and macOS by simply using `cmake` commands¹. No external dependencies were required.

¹As a note, there are no separate projects for different operating systems. This is achieved through the use of compiler directives such as `#ifdef _WIN32`, `#ifdef _UNIX_`, `#ifdef __APPLE__`, and others; each compiler on the operating system will then know which code to compile and which code from other operating systems to ignore. This enables a single source code to be universally compiled for all target operating systems.

3.2. Preliminaries: data sources

An ODBC connector consists of a single library file (`.dll` on Windows, `.so` on Unix-based systems, and `.dylib` on macOS).

Let's see the process of registering a connector in our system, taking the opportunity to introduce concepts that we will continually refer to. The following concepts are shared by all operating systems where ODBC has support.

The fundamental concept is that of a *data source*. A data source abstractly consists of a system registration configured by the connector user (hereinafter, in this work, "the user") through which certain data is accessed. These system registrations (under a name) have attributes that allow for connection to said data origins. In this way, we can register data sources associated with database management systems, or also for other systems like Excel (which offers data sources such as `.CSV` files).

For example, a data source associated with MySQL has attributes such as the path of the associated ODBC connector to be used for the connection, username and password, and other parameters related to network configuration, character sets, etc. In this way, the data source is the entry point to the databases associated with that user account in MySQL, and it establishes that connections are made under the conditions set in said data source.

For the case of DES, the concept of a data source would correspond to a local DES database (recall the schema in Figure 2.4). Thus, we can already see that our data sources associated with DES should reference, in some way, the minimum and indispensable paths of the DESODBC connector, the DES executable, and the working directory.

Continuing with the explanation of the data source concept, for example, we could also want to have two data sources associated with the DES connector, one operating from a certain working directory A, and the other from another working directory B². Both, both conceptly and in the system, they correspond to distinct data sources. For example, an application like MySQL could choose to connect to DES using only the data source corresponding to that of working directory A.

Before creating data sources, we must have our connector registered so that the data source can refer to it. This is done in the `odbcinst.ini` file³, where an ODBC driver is registered by specifying, at a minimum, the path to its library file. Depending on the driver, more parameters associated with the connector's registration are included. In our case, the connector attributes we have considered are:

- **DRIVER**: the path to the connector library (required)
- **SETUP**: the path to the setup library (we will see what this is later).

²Nevertheless, nothing prevents us from registering two identical data sources, with the same attributes, but named differently.

³In Windows it constitutes a log file. In Unix-based systems, these are regular files located in specific locations, depending on the user's installation.

Once the connector is registered, we create an associated data source by registering it in the `odbc.ini` file⁴. Data sources can be of the user type, in which case `odbc.ini` has user permissions, or of system type, that may require elevated privileges. Most commonly, there are different `odbc.ini` files in various locations depending on whether they refer to user or system data sources.

In the first lines of the `odbc.ini` file, data sources are declared: their name and the registered driver they refer to. Within the data source registration itself, the attributes that are required and that we have considered necessary are:

- **Driver:** the path to the connector's library. It is not strictly necessary since we can already know it by knowing which connector it is associated with, but it is standard to include it.
- **DESCRIPTION:** description of this data source.
- **DES_EXEC:** path to the DES executable.
- **DES_WORKING_DIR:** working directory path required from which **DES_EXEC** will be executed.

3.3. Development of the installer

The installer is a command-line executable binary of great simplicity. Commands are provided to install the connector, install a data source with specific attributes, list available data sources, etc. We were able to reuse the code from this application almost entirely, eliminating the possibility of including the attributes related to MySQL and implementing what is needed for a data source for our connector. In Figure 3.1, we can see an example of its use.

```
shell> desodbc-installer -d -a -n "DES ODBC Unicode Driver"
-t "DRIVER=C:\desodbcw.dll;SETUP=C:\desodbcS.dll"
Success: Usage count is 1

shell> desodbc-installer -s -a -n "DES ODBC Test Data Source"
-t "DRIVER=DES ODBC Unicode Driver;
    DES_EXEC=C:\des\des.exe;DES_WORKING_DIR=C:\des"
Success
```

Figure 3.1: Example of connector and data source installation.

3.4. Development of the setup library

We had mentioned in passing that when registering our connector in the system, we could specify a setup library. MyODBC, and therefore our connector, includes

⁴Same considerations as in `odbcinst.ini`.

this library, whose function is to modify the attributes of a registered data source, with the attractive added possibility of being able to call this library at runtime of a client application (`SQLDriverConnect`, as we will see), to specify and/or change the data source attributes dynamically before connecting to it.

We emphasize that we always have the possibility to change the attributes of the data source by modifying ‘`odbc.ini`’, but the configuration library provides us with the ability to do so through a graphical interface (GUI): see for example the GUI we have created for Windows in Figure 3.2.

We proceeded to adapt the configuration library project on Windows. The library is designed based on a resource file (see [10]) that specifies the GUI itself, and a series of C++ source files that interact with it. These files are modified as necessary, aided by the visual editor of .rc files provided by Visual Studio, and the final result can be seen in Figure 3.3.

With respect to the configuration library for Unix-based systems, it is important to note that this type of library is not provided for these systems. Developing them goes beyond the objectives proposed in Section 1.2, where we stated that MyODBC offered a configuration library only for Windows.

However, we should remember that we always have the possibility to manually modify the *odbcinst.ini* and *odbc.ini* files on any system, which is indeed the standard and expected procedure for users. On macOS with iODBC, there is also the possibility to access the *iODBC Data Source Administrator*, which allows for minimally graphical editing of the attributes of any data source.

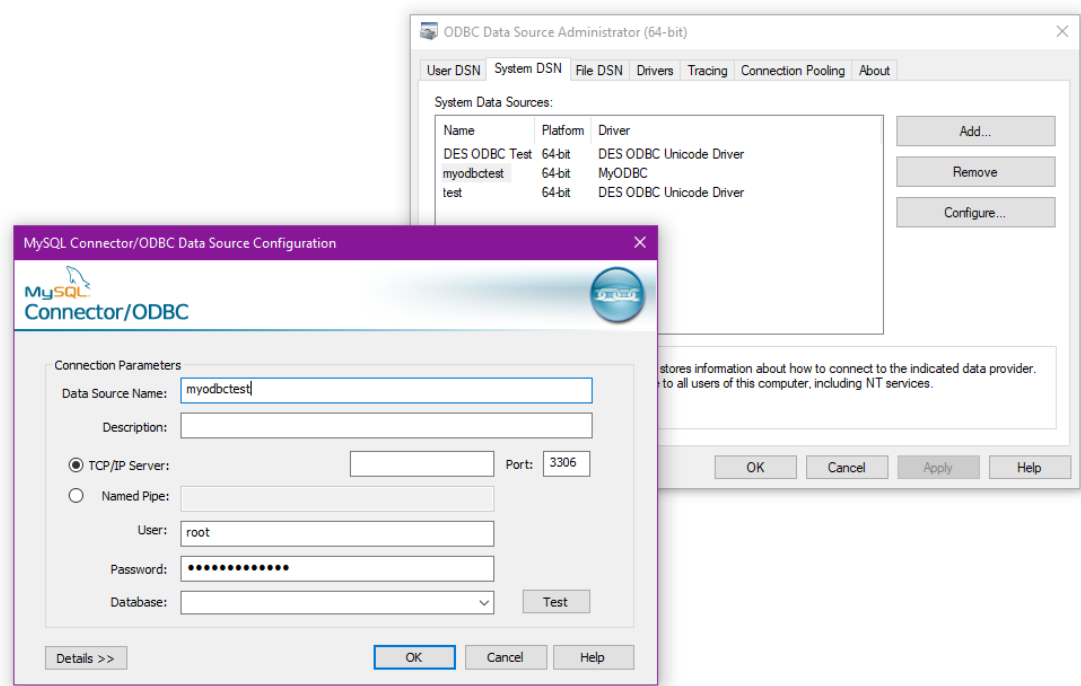


Figure 3.2: Graphical interface provided by the MyODBC library configuration. By clicking on **Configure** a data source in the *ODBC Data Source Administrator* (Windows), it calls to the setup library showing the user a GUI from which the attributes of a data source can be modified.

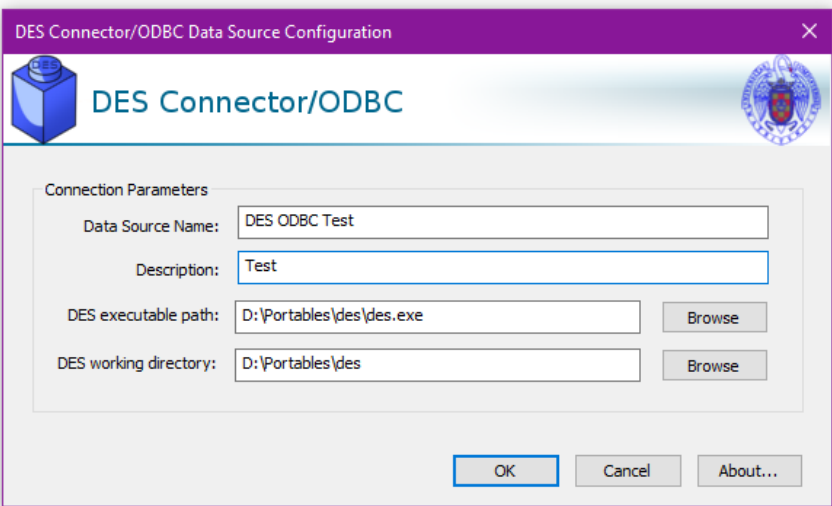


Figure 3.3: Graphic interface provided by the DESODBC configuration library on Windows.

However, although MyODBC does not provide official distributions with configuration libraries for GNU/Linux, the source code allows the compilation of three of them. We have worked on these configuration libraries in an attempt to rescue them: however, we do not guarantee their correct operation, just as MyODBC does not either. After a series of tests, the obtained products and their capabilities are:

- A "minimalist" library that we have named `libdesodbcS.so` (in the MySQL connector, `libmyodbcS.so`), as shown in Figure 3.4. This library provides read and write capabilities for parameters using a standard function `ODBCINSTGetProperties`, which a specific application must call. `unixODBC` refers to a suite of external applications `unixodbc-gui-qt`⁵ on its website, which provides `unixODBC` with a Qt-based GUI interface [11], but must be downloaded and compiled manually. We verified that `libdesodbcS.so` works correctly with this suite.
- Two GUI libraries, one for GTK-2 and the other for GTK-3, named `libdesodbcS-gtk2.so` and `libdesodbcS-gtk3.so` under the same naming policy as before. See Figure 3.5. These libraries do not work correctly under `unixodbc-gui-qt`, and future work will consider providing this compatibility. Nevertheless, an application can execute these libraries when calling `SQLDriverConnect` by requiring the completion of information—the expected behavior is to display the library's configuration interface before establishing the connection.

Thus, although we do not officially guarantee the correctness of these libraries, they may be useful in the contexts we have mentioned.

Regarding the work description, to modify the GUI on Windows we had to modify a resource file; the analogous procedure here is to modify a `.glade` file, which specifies the appearance of the GUI in XML format. This format corresponds to the type of file generated by the Glade tool, a visual editor that allows the development of GTK-type graphical interfaces. With Glade we edited this file, redesigned the GUI to look as similar as possible to our Windows result, and finally exported to GTK-2 and GTK-3.

3.5. Development of the ODBC driver

As we mentioned in the introduction of this chapter, we will describe the implementation while weaving in the necessary theoretical concepts to understand it. The manual that defines the standard and through which we will get to know the different elements of ODBC is Microsoft's documentation [2].

The implementation of an ODBC connector, in essence and being a library, consists of implementing the ODBC API functions. Our philosophy in this document will be to go through the most important functions (it is not necessary to describe the numerous functions of little interest here), which we will classify according to their scope of use.

⁵<https://github.com/mazbrili/unixodbc-gui-qt>

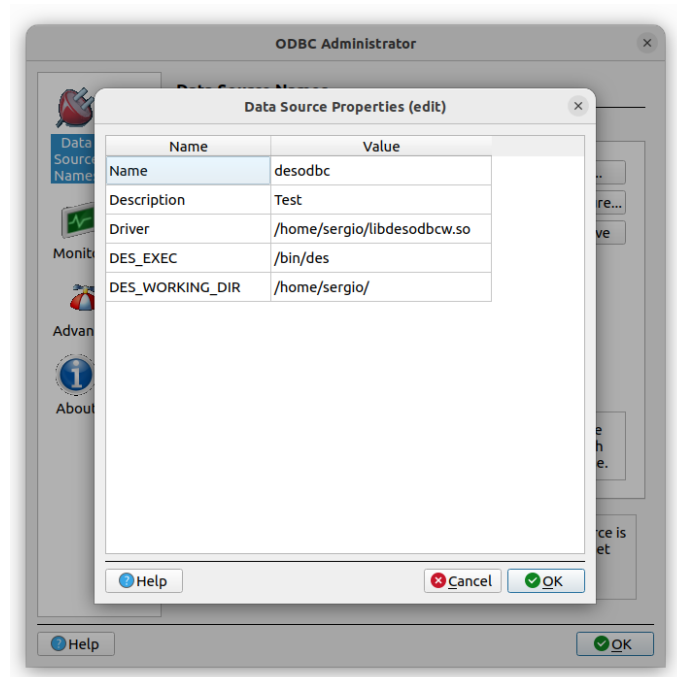


Figure 3.4: `libdesodbcS.so` (desde `ODBCManageDataSourcesQ5`). This data source administrator calls the `ODBCINSTGetProperty` function from the `libdesodbcS.so` library, and the results collected are displayed in a GUI (Data Source Properties (edit) window) provided by the application itself.

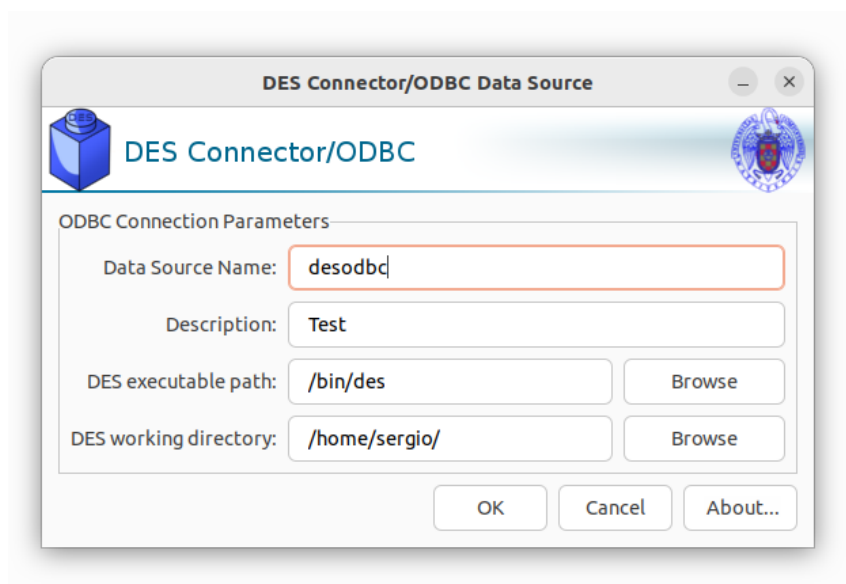


Figure 3.5: `libdesodbcS-gtk2.so` / `libdesodbcS-gtk3.so` executed by a call to `SQLDriverConnect`.

In the ODBC standard, it is not mandatory to implement all functions: ODBC provides a wide range of functions that can be referenced, but each DBMS has unique characteristics that may make certain functions meaningless in some contexts.

This occurs in MyODBC and also in the case of DESODBC: as will be seen in each case, there are functions that we have not implemented due to relevant circumstances. The standard will be that when the user references these functions, a "not implemented" message is displayed.

Similarly, we will only describe the most relevant details of the implementations.

3.5.1. Main classes and getter/setter functions

3.5.1.1. Basic ODBC elements. Handles

We will briefly present the fundamental theoretical concepts in ODBC, which we will refer to throughout this document, and which are essential for understanding its operation the expected behavior of functions from the ODBC standard.

The ODBC API defines a series of elements (or classes, henceforth in terms of implementation) that every ODBC connector must consider in its implementation: environment handle structure (*environment*), connection (*connection*), *statement* (*statement*) and *descriptors* (*descriptors*) [12, p. 87], which we now proceed to explain.

Each connector implements the attributes it deems appropriate in the classes to adapt to the characteristics of the target DBMS; the application simply expects each class to exhibit the behavior appropriate for ODBC standards.

Therefore, since the ODBC Driver Manager and applications are agnostic to the internal implementation of the connector—the components are black boxes outside the connector—all involved parties—application, ODBC Driver Manager, and database—consent the manipulation of some elements called handles, which are 32-bit pointers that point to regions of the connector's ODBC memory map where each respective class instance is hosted [13].

We list the standard handlers from most global to most specific, following the structure and explanations from [12, pp. 87-91]:

Environment handles The environment element is a class that holds global attributes, such as information related to the operating system or the ODBC version that is employed. In the standard, these are usually elements with few attributes and very general.

In our case, the environment element will be implemented in a C/C++ class denoted ENV.

The environment class maintains a list of its associated connection handlers, corresponding to the handler described below.

Connection handles The connection class houses attributes related to a particular connection. This corresponds to the **connection objects** we've already seen in Figure 2.4⁶. In DBMS that use networks, these could be the port, the maximum

⁶As inferred from the referenced figure, the environment elements play no relevant role. They merely contain the connection objects, which are the ones that actually participate in the architec-

size of the packages... Other examples would be the character set used, logs, the current database to which it points, information about the last error, etc.

For future reference, in our case, the connection element will be materialized in a C/C++ class called **DBC**.

The connection class stores a list of its associated statement handlers, corresponding to the handle described below.

Statement handles They refer to the class that stores attributes related to a statement (*statement*), and thus considers all the information related to the complete process of a query; the query string itself, parameters used, results returned by the DBMS, etc. The results we just mentioned are called result sets (*result sets*).

We clarify at this point that the string that the user sends as queries are simply valid instructions according to the target DBMS language; in our case, commands supported by DES.

For future reference, in our case, the sentence element is materialized in a C/C++ class denoted **STMT**.

A class statement stores a list of its associated descriptor handlers, corresponding to the handler described below.

Descriptor handles A descriptor is an object of a class that holds metadata regarding statement objects [12, p. 91].

At this point, it is necessary to briefly introduce the concepts of buffer and parameters.

The application that connects to the connector can declare a series of variables that it can request from the connector to bind to a statement, so that they are updated automatically when executing a query or performing certain operations with it. The standard used as a variable are these *buffers*, specifically the data buffers (*data buffers*) and length buffers (*length/indicator buffers*) [14] (see how they are organized in Figure 3.6). They are usually referred to as application buffers (*application buffers*) because they are created in the application.

For example, if an application wants to link a variable `employee_name` so that the driver, when executing a query, writes to the variable with the information of the `name` column of the final result when navigating through the rows, the application should proportionate these data buffers and lengths so that, when concluding the operation, it can know how to recover the final string from the buffer overwrite addresses and the final length (addresses that have been respected by the driver).

The parameters, on the other hand, are the variables we have just defined but linked to variable parameters in an ODBC statement. For example, we may want to execute a query [15]

```
INSERT INTO employee(name, age) VALUES (?, ?)
```

ture of our connector. It is standard for an application to host only one environment object, but it can accommodate an arbitrary number of them. It doesn't matter where an object connection is stored, whether in one object environment or another. The only important thing is the working directory each connection refers to.

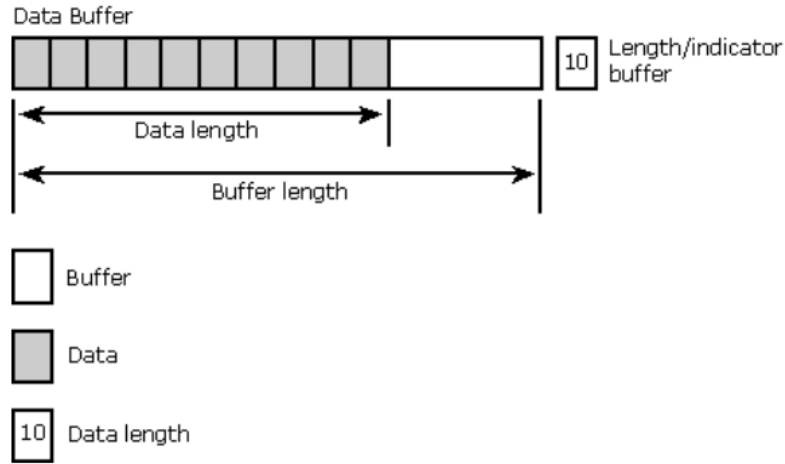


Figure 3.6: Architecture of buffers in the ODBC standard. Extracted from [14].

linking the two parameters with two respective application variables that have already written the relevant information.

We are in a position to define the four types of descriptors defined in the API, quoting them from [12, p. 91] on account of its conciseness:

- *Application Parameter Descriptor (APD)*: “contains information about the application variables (buffers) bound to the parameter markers used in an SQL statement”.
- *Implementation Parameter Descriptor (IPD)*: “contains information about the parameters used in a SQL statement”.
- *Application Row Descriptor (ARD)*: “contains information about the application (buffers) bound to the columns of the result set”. Note that we bind variables to columns: to finally determine the value of the variable, the current row (attribute of the statement object) is used, which the application can manipulate at will through calls to the connector.
- *Implementation Row Descriptor (IRD)*: “contains information about the columns in a result data set”.

For future reference, in our case, the descriptive elements will be objects of a structure in C/C++ denoted as `DESC`.

3.5.1.2. Getter/setter functions

Although we have stated that elements are black boxes for agents other than the connector, it is expected that they will contain a minimum set of attributes that the application can access and manipulate. Getters and setters are the classic procedure that respects encapsulation and the black box concept; consequently, the attributes in question are returned in the format of buffers defined by the standard. The ODBC

API function `SQLGetInfo` is a function that allows us to know the specific values that our connector or the data source we are using has for certain predefined attributes in the standard (for example, the maximum number of users connected at the same time) [16]. We also find the following functions in the following areas:

Diagnosis They refer to errors: `SQLGetDiagRec`, `SQLGetDiagField`, `SQLError`.

Environment Functions `SQLSetConnectAttr`, `SQLGetConnectAttr`.

Statement Functions `SQLSetStmtAttr`, `SQLGetStmtAttr`.

Connection Functions `SQLSetConnectAttr`, `SQLGetConnectAttr`.

Descriptors Functions `SQLCopyDesc`, `SQLGetDescField`, `SQLSetDescField`, `SQLGetDescRec`, `SQLSetDescRec`.

It has been necessary to modify some of these functions that were implemented in MyODBC, for example, in scenarios where some of these attributes refer to capabilities that MySQL and DES do not share (for example, an application might call `SQLGetConnectAttr` requesting the `SQL_LOGIN_TIMEOUT`, a concept that makes no sense in a local/embedded DBMS like DES).

3.5.2. Environment and connection functions

3.5.2.1. Environment

`SQLAllocHandle` is a function that instructs the driver to allocate memory space for a certain type of element (connection, environment, statement, or descriptors) and return its pointer (the handle) [12, p. 140]. This function simply calls the constructors of the corresponding classes and therefore could be reused in its entirety.

3.5.2.2. Connection

The connection process is the first point at which MyODBC would fundamentally differ from DESODBC, since MySQL is a network DBMS and DES is a local/embedded DBMS.

The paradigmatic function for connecting to a data source associated with a connector is `SQLConnect`. In Listing 3.1 we can see its header.

We observe that the client application (MySQL, Access...) previously first hosted space for an environment handler, and subsequently for a connection handler (`ConnectionHandle`) associated with this environment handler.

```

1 SQLRETURN SQLConnect(
2     SQLHDBC ConnectionHandle,
3     SQLCHAR * ServerName,
```

```

4     SQLSMALLINT NameLength1,
5     SQLCHAR * UserName,
6     SQLSMALLINT NameLength2,
7     SQLCHAR * Authentication,
8     SQLSMALLINT NameLength3);

```

Listing 3.1: Header of the function `SQLConnect`. Extracted from [17].

We see that we receive in `SQLConnect` server parameters (`ServerName` and `NameLength1`), user parameters (`UserName` and `NameLength2`), and authentication (usually the password) (`Authentication` and `NameLength3`). The server name corresponds to the installed data source name in the system. DESODBC ignores the user and authentication parameters; there is no built-in authentication mechanism in DES.

The implementation of `SQLConnect` in DESODBC is initially identical to that of MyODBC, and it consists of retrieving the attributes saved in the system relating to the specified data source (in case the data source does not exist, the ODBC Driver Manager itself handles throwing the corresponding error).

Once all the attributes have been stored in a relevant `DataSource` class, which we partially reused from MyODBC, the connection process proceeds.

Another ODBC API connection function is `SQLDriverConnect`. This function is essentially the same as `SQLConnect`, except that it assists the user when they need information or have included insufficient data to make the connection [18]. In our case, as mentioned before, a GUI window associated with our configuration library will be launched in these scenarios, allowing us to dynamically fill in the `DES_EXEC` and `DES_WORKING_DIR` attributes at the client application's runtime (in this way, we can specify these attributes without writing to the operating system's registry).

Lastly, the other connection function of the API is `SQLBrowseConnect`, which assists the user in a more comprehensive manner, but its implementation is complex. MyODBC chooses not to implement it; in our case, it would also not make sense to do so given the simplicity of our data sources.

With respect to the connection itself, we cannot reuse anything from the connection to the system's MySQL server, since the process of connecting to DES is completely different. The task then consists of completely rewriting a connection-handling internal function `connect` that is responsible for the connection process itself. The general operation, common to all versions of DESODBC (both Windows and Unix-based systems such as GNU/Linux and macOS), is illustrated in algorithm 3.1.

Case I. The calling application is the only user currently connected to the working directory (i.e., the number of connected users is zero prior to the call).

1. We create the pipes that will provide us with two-way communication with a global DES process that we are about to create.
2. We proceed to create the global DES process that will serve all applications connecting to DESODBC with the same working directory:
 - a)* Now that we know all the attributes associated with the data source, we consult the `DES_EXEC` attribute, which is the path to the DES executable in the system.
 - b)* We create a DES process from the path and associate the pipes we have created with its input and output.
 - c)* We consume the initial message that DES provides at startup.

Case II. The calling application is not the first user connected to the working directory (there is already a global DES process running from that working directory).

1. We connect to the pipes that were previously created by the first user.

Algoritmo 3.1: High-level process to connecting to DES common to every operating system.

Users can determine if they are the first user by reading a certain attribute of the shared memory. The shared memory will be the way the different connections of the connector communicate in order to know the global state of the DES process, the number of users currently connected, etc.

Access to shared memory is based on the following idea: our application identifies the working directory path by examining the data source and applies a hash function to it, resulting in a unique identifier for that working directory. The file names in shared memory are built with this identifier as a suffix, so that each connection knows exactly which specific files it should access and no others. See Figure 3.7.

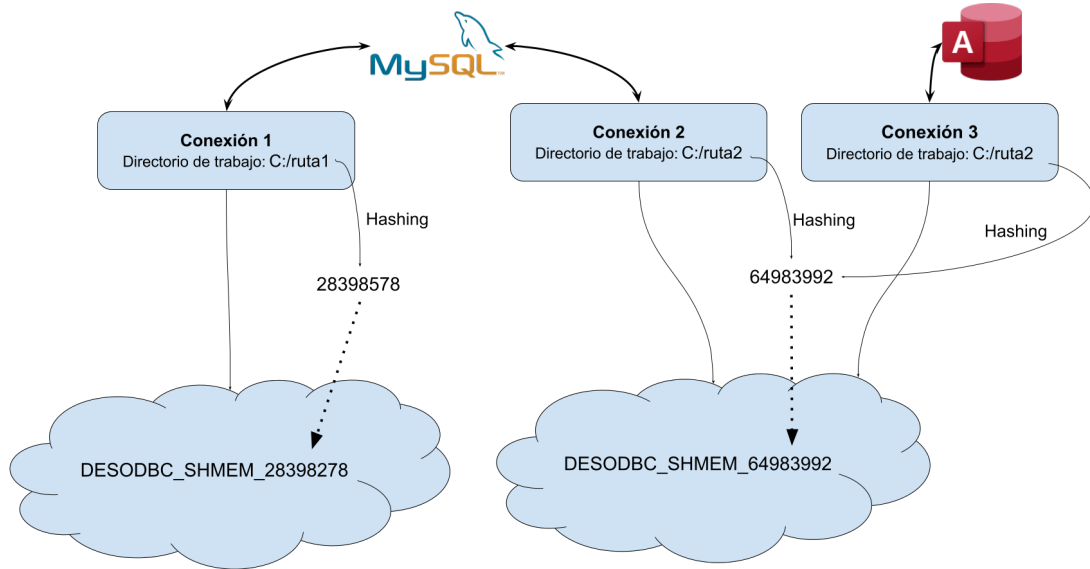


Figure 3.7: Schema of the hashing to access shared memory.

The preliminaries to the algorithm 3.1 consist in knowing the number of connected users. Recall that to access shared memory, we can only know the file names when we apply hashing to the working directory. Therefore, in the connection process, first we collect information from the data source. Next, we build the file names for shared memory IPC and locks (see Listing 3.2).

```

1 // driver/connect.cc (DESODBC)
2 this->connection_hash =
3     std::to_string(str_hasher(des_exec_path_str + des_working_dir_str));
4
5     this->SHARED_MEMORY_NAME = buildName(SHARED_MEMORY_NAME_BASE);
6     this->SHARED_MEMORY_MUTEX_NAME = buildName(
7         SHARED_MEMORY_MUTEX_NAME_BASE);
8     this->QUERY_MUTEX_NAME = buildName(QUERY_MUTEX_NAME_BASE);
9     this->IN_WPIPE_NAME = buildName(IN_WPIPE_NAME_BASE);
10    this->OUT_RPIPE_NAME = buildName(OUT_RPIPE_NAME_BASE);

```

Listing 3.2: Building IPC filenames.

At this point, it is necessary to discuss the IPC mechanism for concurrency. As can be inferred from the previous code, our mutual exclusion system is quite simple, with only two locks: the shared memory lock and the query lock.

Previously, when we addressed concurrency policies, we mentioned that we needed to implement a coarse-grained mutual exclusion solution. For a query to be executed, applications will compete for a single lock; the same applies for reading and writing to shared memory, with another unique lock.

We are presented with two reader-writer problems in both shared memory and the DES console. One of the most refined and efficient solutions to the reader-writer

problem is the so-called witness-passing solution (see [19, section 4.4.3]), whose implementation has considerable conceptual complexity. This solution allows for parallel reading under certain conditions, improving efficiency, which we will not consider in DESODBC since it poses consistency issues. In more detail, our coarse-grained policy is based on the following reasons:

1. Our model of DES input and output does not resemble a database to read or write on. A user should only read the outputs corresponding to the commands they themselves have sent, and no others. It is then necessary to acquire a single lock for the entire process of sending a query and reading the result, in order to consume the output pipeline before releasing the lock.
2. Our shared memory does not exactly follow the database model. It is not a simple array to operate on as assumed by the general R/W model⁷; shared memory includes a series of attributes that are related and logically coherent in specific ways, and therefore the only way to avoid inconsistent states is for instances to acquire a single shared memory lock, preventing others from reading and writing, with the freedom and time they need until they finish the corresponding tasks and shared memory becomes consistent again. We will see some of these processes later.

Regarding the concrete implementation of locks, in the GNU/Linux version, we will not open semaphore files corresponding to the locks (but we will in Windows and macOS, although the former in RAM and the latter on disk), but they will be within shared memory. This is the most desirable alternative to avoid leaving files in the system that we then have to worry about deleting. However, this will not be possible on macOS: although macOS is Unix-based, it specifically does not implement the POSIX function `sem_init` that would enable saving some unnamed semaphores in shared memory: we have no choice but to use named semaphores stored on disk, using `sem_open` and related functions.

Returning to our IPC initialization process, after constructing the names we proceed to initialize these objects, which will be stored in the connection structure (DBC). We use the corresponding procedures from Table 3.1, which lists the IPC functions used in different systems.

The use of Unix functions differs on some occasions in GNU/Linux and macOS, as we have just mentioned. While we previously introduced the concept of handlers and descriptors in the context of ODBC, in operating systems they will take on

⁷In fact, in the case of Windows, shared memory will have an array of connected clients, but this is only one of the attributes that the shared memory structure presents. Since the number of connected clients tends to be very low and limited to 256, it is not worth implementing an efficient R/W solution and thus unnecessarily complicating the code for this single small array attribute among all the attributes. Indeed, there are few clients, and said array is read and written at very specific moments (only during client connection and disconnection processes, which occur separated by seconds, minutes, or hours), and each operation on the array is practically instantaneous due to its small size.

Procedure	Unix	Windows
Create/open shared memory region	<code>shmget</code>	<code>CreateFileMapping</code> (<code>winbase.h</code>)
Adapt shared memory to a predefined structure	<code>shmat</code>	<code>MapViewOfFile</code> (<code>memoryapi.h</code>)
Create pipe	<code>mkfifo</code>	<code>CreatePipe</code> (<code>namedpipeapi.h</code>)
Open pipe	<code>open</code>	—
Create mutex	<code>sem_open/sem_init</code>	<code>CreateMutexEx</code> (<code>synchapi.h</code>)
Acquire mutex	<code>sem_wait</code>	<code>WaitForSingleObject</code> (<code>synchapi.h</code>)
Release mutex	<code>sem_post</code>	<code>ReleaseMutex</code> (<code>synchapi.h</code>)
Close mutex	<code>sem_close</code>	<code>CloseHandle</code> (<code>handleapi.h</code>)
Close pipe	<code>close</code>	<code>CloseHandle</code> (<code>handleapi.h</code>)
Close file descriptor/handle	<code>close</code>	<code>CloseHandle</code> (<code>handleapi.h</code>)
Close view to shared memory	<code>shmdt</code>	<code>UnmapViewOfFile</code> (<code>memoryapi.h</code>) and <code>CloseHandle</code> (<code>handleapi.h</code>)
Destroy shared memory region	<code>shmctl</code>	<code>UnmapViewOfFile</code> (<code>memoryapi.h</code>) and <code>CloseHandle</code> (<code>handleapi.h</code>)
Destroy mutex	<code>unlink</code>	<code>CloseHandle</code> (<code>handleapi.h</code>)

Table 3.1: Procedimientos IPC empleados en los distintos sistemas.

different meanings. The term *handler* will refer in this context to Windows handles (a pointer to a Windows resource), and the term *descriptor* will refer to Unix file descriptors. As can be seen in the table above, Windows handlers and Unix file descriptors will be analogous concepts.

In Unix, we have used `shmget` and related functions instead of `shm_open`, as it allows us to allocate our shared memory segment in RAM rather than on disk, and we access it using a key that in our case, it will correspond to hashing (greater anonymity).

For the most part, the concrete implementation of the processes described in algorithm 3.1 vary greatly depending on whether we are in the Windows version or Unix. We proceed to describe the specific implementation for both situations, com-

menting on what challenges presented themselves to us and why specific decisions were made based on the existing options.

Case Unix (GNU/Linux, macOS) The implementation for both GNU/Linux and macOS is very similar. This is because we call POSIX functions that are supported by these systems (in macOS, not all, as we have already mentioned), and as a result, the code can be easily ported between them. The IPC functions we will use—those related to shared memory, semaphores, and named pipes— vary little among them.

Let's pause for a moment to consider why we have chosen named pipes. The reason for choosing named pipes over anonymous pipes primarily lies in simplicity. Very easy bidirectional communication between processes with named pipes in Unix-like systems for a problem of our characteristics. The final solution will be an architecture very close to what is studied in this computer science curriculum, for example in the classes of Operating Systems.

The truth is that, despite this, it would have been better to use nameless pipes: intuitively, we see that the implementation would be more complex because the DES process only works on specific input and output pipe pairs, and therefore to share them, an exchange of information between instances of the connector would necessarily be required.

However, the price to pay would have been worth it for the extra security that nameless pipes offer us: no pipes are exposed in the filesystem so that other malicious external process can read and write to them.

That is the reason why we would have chose the named pipes path: however, we find that in Unix it is not possible to implement such an implementation without administrator privileges.

This is because a connector process that would like to duplicate/share an unnamed pipe would have to access the file descriptors of external processes, which is not feasible with user's limited permissions -we do not contemplate that our connector will require administrator permissions: it makes no sense-. It would have to request a duplicate from the connector process owner of the pipelines, but the latter cannot do anything about it, since file descriptors are simply inputs to a specific table for each process and, therefore, from an external process. There is no Unix function that allows duplicating file descriptors targeting objective processes (as in Windows we will see).

In summary, using unnamed pipes in our case would, at a minimum, require administrator permissions.

The other possible alternative would be to use Unix domain sockets. We immediately see that this alternative is impossible since we cannot expect DES to open a socket on its own (it is a precompiled executable, we cannot modify it). These considerations regarding sockets also apply to the case of Windows and therefore we will refrain from mentioning it there.

Once the use of named pipes has been justified, let's go through the case study

we presented: for this, we first need to set the initial conditions. Recall that we had already initialized shared memory and locks. Now, just before reaching the casuistry described by the algorithm 3.1, the following must be determined:

1. If there are other ongoing connections in the working directory whose global DES process refers to an executable different from our DES_EXEC.
2. If not (or there are no previous connections, or connections work with the same DES_EXEC as ours), check if it is necessary to open the global DES process or, on the contrary, connect to the same one if it was already open.

In the first step, we simply need to look in shared memory for a value that refers to the path of the executable. If this value is written and refers to an executable different from our DES_EXEC, there is a conflict⁸: our connection is terminated and the user is informed that they must change working directories or refer to the same executable as the one used by previous connections.

In the second step, we simply query a value in shared memory: `des_process_created`. This value is initialized to `false` when the shared memory file is created.

Now we can determine which case we are in:

Case I. *The calling application is the only user currently connected to the working directory (i.e., the number of connected users is zero prior to the call).*

- We create the pipes that will provide us with two-way communication with a global DES process that we are about to create.

We call the POSIX function `mkfifo` for the pipes `driver_to_des_in_wpipe` and `driver_to_des_out_rpipe` via their respective names we obtained with the hashing (see Listing 3.3).

```

1 // driver/connect.cc (DESODBC)
2 ...
3 mkfifo(IN_WPIPE_NAME, 0666)
4 ...
5 mkfifo(OUT_RPIPE_NAME, 0666)
6 ...

```

Listing 3.3: Creation of pipes in Unix-based systems.

- *We create a DES process from the route, and we associate the pipes we have created on its input and output.*

The idea is to call `fork()` so that a child process manages the future DES process, so that this child process eventually launches with `exec1p` replacing its memory map

⁸The reason why this constituted a conflict was studied in section 2.2.3

with that of DES. Let's see it in detail: suppose we are in the child process (listing 3.4).

```
1 // driver/connect.cc (DESODBC)
2 pid_t pid = fork();
3 ... if (pid == 0) {
```

Listing 3.4: Call to `fork()`.

Let's look at the following code listed in Listing 3.5. From the child process, the pipes `IN_WPIPE_NAME` and `OUT_RPIPE_NAME` are opened, but now that we are from the global DES process perspective, these pipes are for reading and writing, respectively. This will be the opposite of the perspective from the connector (the connector writes in `in_wpipe` to pass information to the DES input, therefore, DES must read from this write; and analogously with `out_rpipe`).

```
1 // driver/connect.cc (DESODBC)
2 dbc->driver_to_des_in_rpipe = open(IN_WPIPE_NAME, O_RDONLY);
3 ...
4 dbc->driver_to_des_out_wpipe = open(OUT_RPIPE_NAME, O_WRONLY);
```

Listing 3.5: Opening pipes in Unix-based systems from the child process.

(note that the names of the file descriptors are now conceived from the perspective of DES —`in_rpipe` and `out_wpipe`—).

We just have opened the files respective to the named pipes, but only from the DES'side. How do we associate these file descriptors to the input and output of the DES global process we are just about to launch? We make use of `dup2` so that these named pipes are associated with the current standard input and output of the child process in which we are. This is what is achieved in the listing 3.6.

```
1 // driver/connect.cc (DESODBC)
2 dup2(driver_to_des_in_rpipe, STDIN_FILENO);
3 dup2(driver_to_des_out_wpipe, STDOUT_FILENO);
```

Listing 3.6: Association of pipes to the input and output of DES using `dup2`.

Next, we store the PID of the child process in shared memory (listing 3.7), because this PID will be that of the DES process that will proceed to replace the memory map of the child process (we need the PID in case we need to call `kill` at some point).

```

1 // driver/connect.cc (DESODBC)
2 shm->DES_pid = getpid();

```

Listing 3.7: Saving the DES PID in shared memory.

Finally, we launch the DES process, replacing the memory map of the child process through that of a DES process (listed in 3.8).

```

1 // driver/connect.cc
2 execlp(des_path, des_path, nullptr);

```

Listing 3.8: Execution of the DES process with `execlp`.

Note that with `execlp`, we have only changed the map from the child process memory. This is what allows both the input and output of the child process, as well as its PID, to remain invariant, and therefore are now associated with DES. Conceptually, it is as if our child process now simply *behaves as DES*.

- *The initial message that DES provides at the start is consumed.*

This is the task of the parent process, which, as we recall, corresponds to the perspective of the connector. To do this, it must open the pipes (now from the connector's perspective) and store the file descriptors in the environment structure (see listing 3.9).

```

1 // driver/connect.cc (DESODBC)
2 dbc->driver_to_des_in_wpipe = open(IN_WPIPE_NAME, O_WRONLY);
3 ...
4 dbc->driver_to_des_out_rpipe = open(OUT_RPIPE_NAME, O_RDONLY);

```

Listing 3.9: Pipes opening in Unix-based system from the father process.

Subsequently, we proceed to read DES's initial message by calling the `read` POSIX function over our `dbc->driver_to_des_out_rpipe`: we simply have to read from the output until the read pipe consumes completely.

Finally, we store in shared memory `des_process_created = true` and `n_clients = 1`, and then release the lock associated with shared memory.

Case II. *The calling application is not the first user currently connected to the working directory (there is already a global DES process running from that working directory).*

- *We connect to the pipes that were already created by the first user.*

We simply have to:

1. Obtain the pipes and store their file descriptors in our environment structure —exactly as the parent process did in the previous case—.
2. Update the number of clients in shared memory, incrementing it by one.
3. Finally, release the shared memory lock.

In the disconnect section (??), we will discuss how all these IPC variables come to an end, whether the user wants to close them by calling the corresponding functions or if the connection is severed abruptly by system signals / power cut.

Case Windows In this section, we will delve further into the IPC (Inter-Process Communication) aspect that Windows implements; this knowledge, unlike that of Unix, is not currently covered in computer science curricula.

Upon contemplating the conceptual simplicity of the solution Unix offers us for a problem like ours, the idea is to what extent we can replicate it in Windows. Let's undertake the entire process in order.

The first difference is that Unix file descriptors (`int`) are translated to handles (`HANDLE`), which are pointers to a memory area, in line with the concepts already seen. In Table 3.1, there is no need to comment on the correspondence between shared memory and locks between Unix and Windows systems, as in this case it is direct.

The IPC aspect that will define a major break between the proposed architecture for connection in Unix and that of Windows is pipes.

Technically, Windows provides a solution for named pipes [20], establishing a two-way communication between what they call a *pipe server* and *pipe clients*.

The functions of the *pipe server* are:

1. Creating a named pipe (`CreateNamedPipe` —`winbase.h`—)
2. Wait for client processes to connect to a named pipe instance (`ConnectNamedPipe` —`namedpipeapi.h`—).

The primary function of the *pipe client* is to establish a connection with the server process through the named pipe that has already been created, either using `CreateFile` (`fileapi.h`) or `CallNamedPipe` (`winbase.h`).

The question that we may ask to ourselves is: who must be the *pipe server*? Who has the characteristics to do it? DES by itself does not have the possibility to act like a server: it is an already compiled executable and that we cannot modify.

If DES cannot be a *pipe server*, it must then be some connection, and of course, it should always be the first one, at least initially —otherwise, it would never be possible to connect—. But what happens if the first user disconnects and there are other active clients? The options are:

- a) That it is predefined that all connections act as servers.
- b) Pass the *batton* role of the server to another user.
- c) An external global executable acts as the *pipe server*.

The option a) is problematic because while Windows allows multiple servers to coexist, from the server, you obtain a unique server-end handle from the named pipe, and therefore, it is not shared between servers. This is an issue because DES only has the ability to assign specific input and output handlers, and change them at runtime, is not an option for DES as it would minimally require administrator permissions (as we will see shortly).

Option b) is also problematic because the user may have abruptly terminated the program. Therefore, the pipe server should send duplicate server-end handles as new users connect, so that they have them available if the pipe server disconnects. But this requires implementing a message exchange system too complex: it is a possibility, but as we will see next, unnamed pipes offer a somewhat simpler solution, but above all, as we have already mentioned for Unix, it is more secure since it does not expose a named pipe to the view of potentially malicious other processes.

The option c) listed previously is not viable: the ODBC connector must consist of a single .dll file.

Let's discard the use of named pipes due to:

- Complex implementation.
- Lack of security.
- There is a possibility that named pipes are blocked by the Windows firewall⁹.
- Unnamed pipes are safer and their implementation is somewhat simpler (and possible without administrator permissions, as we will see now).

Let's observe the rest of the possible solutions:

- a) Use potential input and output handlers exposed by the DES process.
- b) Use nameless pipes.

In principle, option a) is a possibility. We can observe the DES handlers using the software *Process Explorer v17.06*¹⁰ and see that, in principle, there are exposed input and output handlers (see Figure 3.8).

⁹There are several forum entries where users report how the Windows firewall blocks named pipes in certain contexts, mainly due to features of the user's machine—that is, something that we cannot control from the connector—. See, for example: <https://superuser.com/questions/462443/what-are-reasons-for-local-windows-named-pipes-to-fail> (retrieved April 16, 2025).

¹⁰<https://learn.microsoft.com/en-us/sysinternals/downloads/process-explorer>

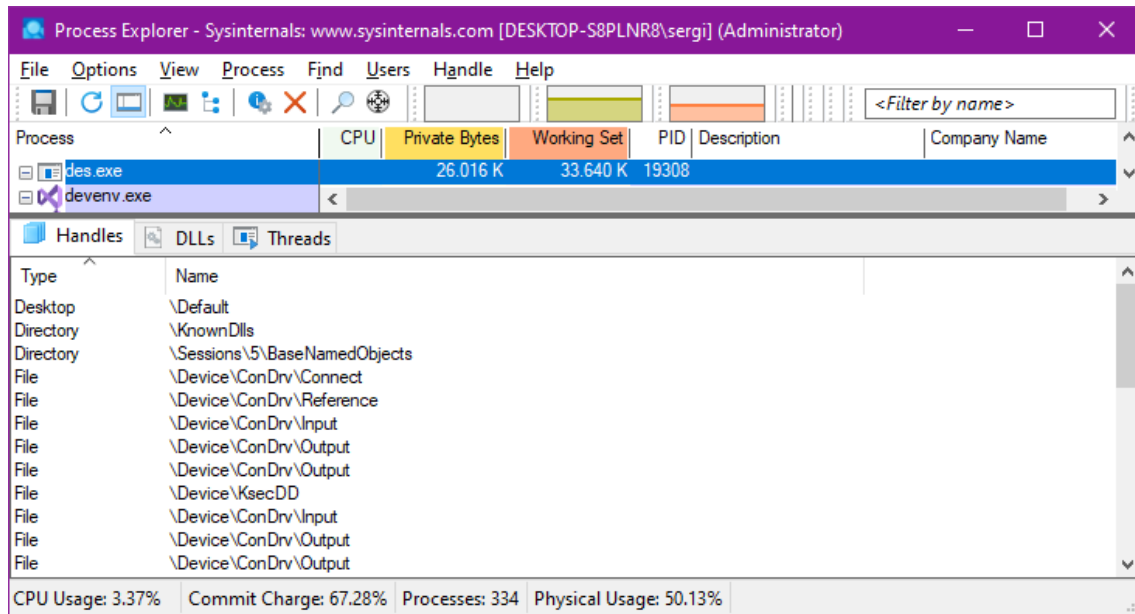


Figure 3.8: View of the handles exposed by DES in the *Process Explorer* software.

However, to have accessed the handles, we already warned in Figure 3.8 in the window title that it has been necessary to run *Process Explorer* in administrator mode. Indeed, certain permissions granted to administrators are required to open handlers from external processes and have complete access to them¹¹; additionally, we wish for the system to be opaque to the connector, with the only exposed points being the specific shared memory and lock files.

Only option b) is left for us. Is it possible for multiple connector processes to access unnamed pipes associated with DES? It seems that this is precisely the situation conducive to named pipes, and therefore, unnamed pipes cannot be shared between multiple processes. However, we have designed an architecture that allows solving all the obstacles encountered using unnamed pipes.

The central idea is that new clients who connect should ask another connected client to generate a duplicate of the handlers. For this, the client requested to copy calls `DuplicateHandle` with the PID of the target process that will use the copy (this is necessary because the copy is specific and exclusive to each process, there are no "global copies"). Therefore, there is an exchange of information between the process that requests the copy and the process that generates it, only within the confines of shared memory.

This exchange of information in the copying process motivates the use of *event objects* [22], which are synchronization objects that send/receive signals, indicating that a certain event has taken place. We distinguish three events: request for copy from the handler (`request_handle_event`), «copy of the handler sent» (`handle_sent_event`) and «request to close connection» (`finishing_event`). We

¹¹"To open a handle to another process and obtain full access rights, you must enable the *SeDebugPrivilege* privilege." [21].

will see in what contexts each of these events is used.

We are in a position to start the tour presented in Algorithm 3.1. The background is that we have been able to initialize shared memory and locks. Now we check the preliminaries (existence of other connected clients and possible conflicts) as we did in Unix, and then we can start with the case analysis:

***Case I.** The calling application is the only user currently connected to the working directory (i.e., the number of connected users is zero prior to the call).*

- *We create the pipes that will provide us with two-way communication with a global DES process that we are about to create.*

We create unnamed pipes for input and output (see Listing 3.10).

```

1 // driver/connect.cc (DESODBC)
2 ...
3 ...CreatePipe(&dbc->driver_to_des_out_rpipe, &dbc->driver_to_des_out_wpipe,
4               &des_pipe_sec_attr,
5               des_pipe_buf_size)
6 ...
7 ...CreatePipe(&dbc->driver_to_des_in_rpipe, &dbc->driver_to_des_in_wpipe,
8               &des_pipe_sec_attr, des_pipe_buf_size)

```

Listing 3.10: Creating unnamed pipes in Windows.

- *We create a DES process from the route, and we associate the pipes we have created on its input and output.*

In Windows we can create a process with `CreateProcess` (`processthreadsapi.h`). The key functionality that we will use is that in the parameters of that function we can send a `STARTUPINFO` structure, and in whose interior we can establish the input and output handlers that will be associated with the process to execute. We then limit ourselves here to assign to them the ends of the pipes we had initialized previously. The complete process can be observed in Listing 3.11.

```

1 // driver/connect.cc (DESODBC)
2 ...
3 ZeroMemory(&dbc->startup_info_unicode, sizeof(dbc->startup_info_unicode));
4 ...
5 env->startup_info_unicode.hStdError = dbc->driver_to_des_out_wpipe;
6 env->startup_info_unicode.hStdOutput = dbc->driver_to_des_out_wpipe;
7 env->startup_info_unicode.hStdInput = dbc->driver_to_des_in_rpipe;
8 ...
9 ... CreateProcessW(NULL, des_path, NULL, NULL, TRUE, DETACHED_PROCESS |

```



```

10         CREATE_NEW_PROCESS_GROUP, NULL, des_working_dir,
11         &dbc->startup_info_unicode,
12         &dbc->process_info)
13     ...
14 //We save the PID of the DES global process
15 dbc->shmem->DES_pid = dbc->process_info.dwProcessId;

```

Listing 3.11: Creation of the DES process in Windows.

- *The initial message that DES provides at the start is consumed.*

We simply call the function `ReadFile (fileapi.h)` with the handler `env->driver_to_des_out_r` while maintaining the same message reading process as initially described for the Unix case. The following steps are taken to conclude the entire process:

1. `des_process_created` from shared memory is set to `true`.
2. We store the PID associated with the connector instance in the list of active shared memory connections.
3. A thread is created and run in the background, associated with the duplication and sharing of handlers.
4. The shared memory lock is released.

In the following case, we address how handlers are duplicated and shared.

Case II. *The calling application is not the first user currently connected to the working directory (there is already a global DES process running from this working directory).*

- *We connect to the pipes that were previously created by the first user.*

The terminology we will use is *petitioner* process for the process that requests to have its handlers duplicated, and *petitionee* for the process responsible for duplicating and sharing the handlers.

We are in the perspective of the *petitioner* process. It obtains the shared memory locks and `request_handle_mutex`, and then does the following:

1. Puts in shared memory, in a `pid_handle_petitioner`, its PID.
2. Proceed to decide who will be the *petitionee*. For this, it takes a random PID from the list of active PIDs (if this does not respond, try again with another random one), and places it in a shared variable `pid_handle_petitionee`. Releases the memory lock, sends the `request_handle_event` signal, and proceeds to wait for the `handle_sent_event` event, as that is when the duplicate handlers are ready for him.

We switch to the perspective of the *petitionee*. The *petitionee*, prior to all this, runs a thread in which it periodically waited for a call. It receives a `request_handle_event` signal due to the petitioner and examines shared memory to see if `pid_handle_petitionee` corresponds to its own PID: if not, it continues waiting for future events. If it turns out to be the requested process, it proceeds to duplicate the handlers in a specific way for the requestee (with its PID) and place them in shared memory. See Listing 3.12.

```

1 // driver/connect.cc (DESODBC)
2
3 HANDLE petitioner_process_handle =
4     OpenProcess(PROCESS_DUP_HANDLE, TRUE,
5                 shmem->handle_sharing_info.pid_handle_petitioner);
6
7 ... DuplicateHandle(GetCurrentProcess(), dbc->driver_to_des_out_rpipe,
8                     petitioner_process_handle,
9                     &shmem->handle_sharing_info.out_handle, 0, TRUE,
10                    DUPLICATE_SAME_ACCESS);
11
12 ... DuplicateHandle(GetCurrentProcess(), dbc->driver_to_des_in_wpipe,
13                     petitioner_process_handle,
14                     &shmem->handle_sharing_info.in_handle, 0, TRUE,
15                    DUPLICATE_SAME_ACCESS);

```

Listing 3.12: Duplicate of pipe handlers from the *petitionee* process to the *petitioner* process.

Next, it activates the `handle_sent_event` event so that the *petitioner* knows they can pick up their new handlers.

We return then to the perspective of the petitioner, who now proceeds to store in their 'ENV' structure the shared memory handlers, clean the structure of a 'HandleSharingInfo' type for future share, and release the locks. This process can be seen in the listing 3.13.

```

1 // driver/connect.cc (DESODBC)
2 ...
3 dbc->driver_to_des_out_rpipe = dbc->shmem->handle_sharing_info.out_handle;
4 dbc->driver_to_des_in_wpipe = dbc->shmem->handle_sharing_info.in_handle;
5
6 // we reset the structure once we have saved the handles
7 dbc->shmem->handle_sharing_info.in_handle = NULL;
8 dbc->shmem->handle_sharing_info.out_handle = NULL;
9 dbc->shmem->handle_sharing_info.pid_handle_petitionee = 0;
10 dbc->shmem->handle_sharing_info.pid_handle_petitioner = 0;
11 ...

```

Listing 3.13: Receiving handles for pipes and cleaning up the shared memory structure `HandleSharingInfo` by the *petitioner*.

3.5.2.3. Disconnection

The standard and correct way to disconnect is by calling `SQLDisconnect`. However, robust and secure code should also cover cases where applications close under other circumstances. We will see the two possibilities.

SQLDisconnect The `SQLDisconnect` function takes the connection handler as an argument, and its abstract goal is to close the connection [23]. Our interpretation is that after this call, the handler still points to our DBC structure, but it no longer has valid pointers to IPC resources. To destroy the DBC structure the function ‘`SQLFreeHandle`’ must be called with the connection handle in question.

In this function, if applicable, we will take care of releasing the shared memory, locks, pipes, and possibly the DES process. This is why it constitutes a secure connection closure, and it is standard in ODBC to call this function to disconnect. It is important to note from the outset: in both Windows and Unix, closing,^a file handle/descriptor translates to releasing a user from that file", so that those resources still exist for other connected clients. However:

1. In Windows, when closing a handle (`CloseHandle` —`handleapi.h`—), the object in the system to which it refers is not destroyed, but it is when no handle pointing to it remains [24].
2. In Unix, closing a file descriptor (`close`) does not destroy the object in the system to which it refers, even if there are no open file descriptors remaining. To destroy it, you must call some function as `unlink`.

Later we will analyze in detail this behavior and the consequences that imply when an insecure closure occurs.

In ‘`SQLDisconnect`’, before disconnecting, we examine the shared memory to see how many clients are connected. We distinguish the following cases:

a) There are more clients

So, based on our analysis, we simply need to call the *close* of the resources without further complication. It is worth noting that in the case of Windows, since we have to manually close our pipe-sharing thread, running perpetually in the background, if we don’t want the application to hang indefinitely. The entire procedure in Windows would be:

1. The current PID is placed in shared memory as *petitioner*.
2. A closing signal (the `finishing_event` we listed previously) is sent.
3. On any client, the thread sharing the pipes captures this signal, and checks in shared memory the PID of the petitioner. If it matches its PID, it is considered "summoned" and exits the infinite wait loop, allowing the thread to close.

4. The pipe handles are proceeded to be closed. Remember that as long as there is another handle pointing to the pipes, Windows will not delete them.
5. The client removes its information from shared memory: it is now no longer available to other clients.
6. It closes its handlers to shared memory and locks.

b) We are the last remaining client

When the last user is disconnected from the work directory, we first have to

- Terminate the DES process. To do this, send the command `DES /q`. This is because, in addition to being a secure closure, it may not be possible to call `kill` (Unix) or `TerminateProcess` (Windows, `processthreadsapi.h`) due to permission issues.
- Close the file handlers/descriptors.
 - In the case of Unix, also destroy these resources with `unlink`.

Let's see what happens if the connector is closed without calling `SQLDisconnect`.

Abrupt closure of the driver. Windows and Unix When an application that has hosted a connection handler abruptly terminates without calling `SQLDisconnect` or releasing the connection handler, the ODBC Driver Manager also does not perform these steps [25; 26]: remember that the ODBC Driver Manager is not a separate process, but a library loaded into the application's memory space.

What has happened is that when the application abruptly ended, all its memory space was simply destroyed: the ODBC Driver Manager, the connector, the memory to which the handlers point... but all this, without having executed any routine that would have allowed us to have performed a "safe closure".

Then, we know that ODBC resources are destroyed along with the application. But we must ask ourselves, what happens with the resources related purely to DES—pipes, our shared memory, locks, and the DES process—? We cannot allow our DES-related resources to be neglected and scattered throughout the system. We cannot rely on the application always closing properly, either due to poor programming or an unexpected termination of the application process.

Windows assists us in this task: by default, it destroys shared memory files and pipes when no process is accessing them¹².

¹²In [27], it is explained that there is a parameter `OBJ_PERMANENT` to force that object do not self-destruct, and that, by default, *such objects are deleted when all open handles to them are closed*.

The case of what happens with the DES process is special. At first glance, it would seem logical that Windows has no reason to end itself. Indeed, it can be verified with a test application that if it calls a simple `CreateProcess` (`processthreadsapi.h`) and immediately thereafter the application terminates, the called process continues to run.

However, in our connector, prior to calling `CreateProcess` (`processthreadsapi.h`), we set its `STARTUPINFO` structure to initialize the standard input, output, and error handles with associated pipes. It appears that when the pipes cease to exist, the process proceeds to terminate. We have not found documentation that supports this behavior is not provided in the given text, but an experiment is described involving the use of `CreateProcess` from `processthreadsapi.h`. The experiment suggests that when using `CreateProcess` to create a new process, the behavior of the process depends on whether pipes are connected or not. In one case, where pipes are connected, the process ends when the application ends (*ceteris paribus*). In the other case, where pipes are not connected, the process does not terminate with the application.

In summary, we have checked empirically¹³ that Windows takes care of freeing up resources for us. The explanation for this is that we believe that, after an application is abruptly terminated when it is the only connection to a global DES process, the following occurs:

1. Windows detects that there is no open handler associated with the shared memory and pipes.
2. Therefore, Windows proceeds to destroy these objects.
3. The DES process encounters that its `stdin/stdout` correspond to destroyed pipes.
4. The DES process subsequently comes to an end.

Let this explanation be precise or not, in any case, we do not have to make any additional modifications, since we know for certain that Windows releases the appropriate resources. In the extreme case of a power outage, there is no problem: since all instances of the connector run on the same machine, all connections will be terminated equally. As a final note, we point out that in Windows, if the application is closed without calling ‘`SQLDisconnect`’, even though it has not been able to remove itself from the shared memory list (as described in the implementation of ‘`SQLDisconnect`’), when another client requests to share its pipes will receive a *timeout*, and then proceed to remove this ghost client from the list.

Unfortunately, Unix will not facilitate a safe closure for us. If an application is closed not safely and it was the only remaining connection to a certain DES process:

¹³Among other utilities, we have used `AccessChk` for that purpose. In section (3.8) we look into this more profusely.

- The shared memory file created from `shmget` does not get removed from RAM.
- Pipelines remain on disk, and in the case of macOS, locks have the same fate¹⁴.
- The DES process will remain active (after performing `fork()` and the parent process destroying itself, the orphaned child process —DES— is adopted by the process above the parent in the hierarchy, and there will always be one (either `init` or one of its children).

Fortunately, these scenarios are rare: they would occur when there is a power outage or when the `SIGKILL` signal is sent to the application. When `SIGKILL` is sent, the process is immediately destroyed: the system does not allow this signal to be captured.

However, a user should not resort to sending `SIGKILL` unless absolutely necessary. In fact, the correct course of action in most situations is to send `SIGINT` or `SIGTERM` signals to allow for a safe shutdown of the application, enabling it to save any necessary data and perform cleanup operations: that is because these signals, unlike `SIGKILL`, can be caught and handled, ensuring that files and connections are properly closed before the process is terminated.

The a priori solution would be for our connector to intercept these signals and manage them. This presents the problem that it may interfere with the user application in case it also intercepts these signals. We opted to add the `atexit` directive to our connector, which allows calling a specific function in case the application does not close unexpectedly, and then proceeds to destroy the loaded library [28]. See Listing 3.14.

```

1 //driver/connect.cc (DESODBC)
2
3 void safe_close_connections() {
4     for (auto hdbc : active_dbcs_global_var) {
5         hdbc->close();
6     }
7 }
8
9 __attribute__((constructor))
10 void init() {
11     atexit(safe_close_connections);
12 }

```

Listing 3.14: Forced safe closure with `atexit`.

This code has the same effect as calling `SQLDisconnect` on all active connection handles.

¹⁴Recall what was mentioned about macOS: when not implementing the `sem_init` function, it should called named semaphores, stored on disk. However, in GNU/Linux, we introduce (unnamed) semaphores in the shared memory itself.

However, if the connector has been closed in a non-safe manner and the user attempts to open resources already created in previous connections, an error message will be displayed, instructing the user to delete manually the relevant resources.

3.5.3. Statement handling functions

In handling statements is where the primary functionality of an ODBC connector is deployed: managing statements by acting as an intermediary between the application and the target DBMS. We can classify the following API functions in this area: `SQLBindParameter`, `SQLCancel`, `SQLCloseCursor`, `SQLDescribeParam`, `SQLExecute`, `SQLExecDirect`, `SQLFreeStmt`, `SQLGetCursorName`, `SQLNumParams`, `SQLParamData`, `SQLParamOptions`, `SQLPrepare`, `SQLPutData` and `SQLSetCursorName`.

These functions are used to prepare and send statements; the functions that deal with the obtained results will be studied in Section 3.5.4.

Fortunately, we will be able to reuse a large part of the code related to these statement handling functions. MyODBC provides a main class `STMT`: a pointer to an object of this class constitutes a statement handler. Within this class, one can find the expected elements, such as lists of the various descriptor handlers, linked attributes and parameters, string containing the query itself, etc.

Our task is to reuse as much of this class as possible and remove the elements that are strictly related to MySQL. We observe that during the process of query within the API functions responsible for execution, MyODBC only contacts MySQL at the precise moment of sending the query, constituting a single external library helper function that is limited to sending the query to the MySQL Server. In Listing 3.15, a point in the connector code where a query is about to be launched can be observed.

```
1 // driver/execute.cc (MyODBC)
2 ...
3 154: if (!get_result_metadata(stmt, FALSE))
4 ...
```

Listing 3.15: Example of where the query is sent to the MySQL Server in MyODBC.

This function `get_result_metadata` makes a call to an external library associated with MySQL, which, by collecting certain attributes of the statement object, proceeds to communicate with the database in our system. The specific external function called is `mysql_store_result`, whose header can be seen in Listing 3.16.

```
1 // include/mysql-8.0/mysql.h (MyODBC)
2 468: MYSQL_RES *STDCALL mysql_store_result(MYSQL *mysql);
```

Listing 3.16: Header of the `mysql_store_result` function in MyODBC.

We see that the only thing that interests us is that it returns a pointer to a structure `MYSQL_RES`. In Listing 3.17 we can observe its attributes.

```

1 // mysql.h (MyODBC)
2 typedef struct MYSQL_RES {
3     uint64_t row_count;
4     MYSQL_FIELD *fields;
5     struct MYSQL_DATA *data;
6     MYSQL_ROWS *data_cursor;
7     unsigned long *lengths; /* column lengths of current row */
8     MYSQL *handle; /* for unbuffered reads */
9     const struct MYSQL_METHODS *methods;
10    MYSQL_ROW row; /* If unbuffered read */
11    MYSQL_ROW current_row; /* buffer to current row */
12    struct MEM_ROOT *field_alloc;
13    unsigned int field_count, current_field;
14    bool eof; /* Used by mysql_fetch_row */
15    /* mysql_stmt_close() had to cancel this result */
16    bool unbuffered_fetch_cancelled;
17    enum enum_resultset_metadata metadata;
18    void *extension;
19 } MYSQL_RES;

```

Listing 3.17: The MYSQL_RES structure in MySQL.

We observe that, by appropriately filling this structure, our connector does not need more to know the result of the query. Therefore, our task is to reuse the maximum number of attributes from `MYSQL_RES`, eliminate any that are superfluous, and appropriately fill them with the values resulting from the queries.

Studying the source code of MySQL Server, where all these external library functions are implemented, is how we have been able to understand how the attributes are expected to be filled and thus guarantee the same postconditions for the same preconditions.

part related to the raw results corresponds to the attribute `struct MEM_ROOT *field_alloc`. The central idea is to store the results produced by DES in an internal analog structure (`ResultTable* internal_table`) after extracting the information from the TAPI results obtained from a reading of the DES output. The rest of the attributes are also conveniently populated based on these obtained results.

In Listing 3.18 one can observe how the process of executing a query would be rewritten for DES.

```

1 // driver/execute.cc (DESODBC)
2 ...
3 full_query = "/tapi " + query + '\n';
4 ...WaitForSingleObject(env->query_mutex, MUTEX_TIMEOUT);
5 ...WriteFile(stmt->dbc->driver_to_des_in_wpipe, full_query_arr,
6             strlen(full_query_arr),
7             &bytes_written, NULL)

```



```

8 ...ReadFile(stmt->dbc->driver_to_des_out_rpipe, buffer,
9             BUFFER_SIZE - sizeof(CHAR), &bytes_read, NULL);
10 ...ReleaseMutex(env->query_mutex);
11 ...stmt->last_output = tapi_output;
12 ...if (!get_result_metadata(stmt, FALSE)) {
13 ...

```

Listing 3.18: Complete query execution process in DESODBC.

(code presented for illustrative purposes: the actual code is organized differently)

That is:

1. We construct the full query: `/tapi` + user query, to obtain results in TAPI mode.
2. Acquire the query lock.
3. Send the query to DES.
4. Read the output generated by DES.
5. Release the query lock.
6. Store this output within the statement structure, and call the function `get_result_metadata()` (which no longer includes sending the query to MySQL) to construct the final result.

This latter function is the one we have to write from scratch to achieve the same postconditions as a call to `mysql_store_result()`. We would expand too much if we described the complete process of building the internal table in this document, whose sole objective is to present the general operation of the driver.

We have managed to replace the role of MySQL with that of DES in handling a query. However, we still need to address the management of the results themselves, which we will see in the next section. In MyODBC, when handling results, calls are made to external library functions that manipulate the `MYSQL_RESULT` structure. Again, we will replace them with homologous functions with the same behavior.

3.5.4. Result handling functions

Once a query is executed using the relevant API functions, applications read and manage results through certain functions, classified under the "handling results" label, corresponding to `SQLBindCol`, `SQLBulkOperations`, `SQLColAttribute`, `SQLColAttributes`, `SQLDescribeCol`, `SQLExtendedFetch`, `SQLFetch`, `SQLFetchScroll`, `SQLGetData`, `SQLMoreResults`, `SQLNumResultCols`, `SQLRowCount`, `SQLSetPos` and `SQLSetScrollOptions`.

We will not comment extensively on these functions as they may have been fully reused from MyODBC. We will discuss some important aspects we have modified, distinguishing between two approaches to handling results: reading and modification.

3.5.4.1. Reading

Some of the cited functions have a reading"functional objective; that is, to obtain data on the set of results to display to the user.

Cursores We introduce the concept of cursor beforehand. The concept represents an abstract pointer that points to a row in the result set, similar to a text cursor in office software [12, p. 452]. Result set handling functions use the cursor to perform their tasks. The ODBC standard distinguishes the following types of cursors:

- *Forward-only*: they can only move forward in the result set. If you need to access a previous row, you would have to restart the cursor [12, p. 452].
- *Scrollable*: They allow access to any row in the result set regardless of direction [12, p. 451].
 - *Static*: with this cursor *the result set appears to be static* [12, p. 542]; that is, changes made by other connections do not alter the result set perceived by the cursor.
 - *Dynamic*: these are the opposite of static cursors; changes made by other connections to the result set are updated in the cursor's view [12, p. 543]. They are expected to be very costly to implement.
 - *Keyset-driven*: these are dynamic in that they detect updates to the result set values, but they maintain the structure statically based on a key system (keys) of the result table. See [12, p. 543].
- *Block*: Rather than a separate cursor category from the above, it is an attribute of the cursor: if they point to a row block in the result set, they allow for reading a block of rows in one go, thereby improving performance in specific scenarios [29; 30].

It is not necessary to implement all cursors. For example, MyODBC supports *forward-only*, *static*, *dynamic* cursors and block-based retrieval, but does not implement *keyset-driven* cursors.

DESODBC will only implement the purely static cursors: that is, forward-only, static, and the possibility of "block". This type of cursors allows reusing a large part of the MyODBC code related to them, as they are not dynamic, do not call functions from an external MySQL library.

Allowing some degree of dynamism in cursors would exceed the scope of a final degree project, as while it is a valuable feature, it complicates the implementation and could divert attention from the core objectives of the project; it could be carried out with the knowledge of the current curriculum, but the time to invest in this particular functionality would be unreasonable. In fact, its implementation is not mandatory and some ODBC connectors choose not to implement it, thereby forgoing dynamic cursors and keyset-driven ones. In the first case, this is known by examining the source code, in the second by reading the documentation: [31, section

5.4]. Despite this, not including them, we will still meet our proposed minimum objectives. For now, we must remove all MyODBC code related to dynamic cursor types.

Now that we understand the concept of a cursor, the functions related to reading will be based in the movements of this cursor and the returning results that it will find. As before, the philosophy would be to reuse all the code from MyODBC, uniquely reimplementing functions that substitute the MySQL external library functions. we see an illustrative example below.

The `SQLFetch` function is responsible for positioning the cursor on the next row/set of rows of the result obtained from the last statement. In the MyODBC implementation, the `STMT` structure is conveniently manipulated, a process which we will replicate in full, and at a given point in time we call the external library function `MYSQL_ROW STDCALL mysql_fetch_row(MYSQL_RES *result)` with an analogous function that satisfied the same postconditions for the same preconditions. We were aided by the MySQL Server source code, which includes the implementation of this external library function.

Note on markers ODBC offers the possibility of working with bookmarks. A bookmark is simply defined as a value that identifies a row in the result set. In other words, the concept analogous to the name "bookmark" itself would be a bookmark on a page in a book, but in the case of ODBC, it would involve storing "a page number", remembering it, and reading it so that the cursor is positioned on "the page" (the row) when the time comes [32].

Thus, the application can request that the connector place a marker on the row where the cursor is located over the result set, and later request that the connector execute a certain operation on the remembered row: update it, delete it, position itself on it, etc. (Note that the application is only requesting to work by markers: the value and meaning of the marker itself are only known to the connector).

Markers are considered an advanced feature for an ODBC connector¹⁵. We introduce this concept because we will need to refer to it later, and at this point, we simply comment that DESODBC also implements it, as the use of markers does not rely on external MySQL library functions that we have not replaced upfront.

3.5.4.2. Modifying

Once we have a set of results from executing a query, we not only have the ability to read from this set, but also to modify it and send the modifications to the DBMS. The API functions where these operations can be performed are `SQLSetPos` and `SQLBulkOperations`.

¹⁵An ODBC connector does not have to implement it: a user must call `SQLGetInfo` under the `SQL_BOOKMARK_PERSISTENCE` option to check it. In [12], markers are discussed in Chapter 11, *Retrieving Results (Advanced)*; they are considered an advanced feature.

SQLSetPos `SQLSetPos` admits a row number and certain operations to be performed on it as arguments: `SQL_POSITION`, `SQL_REFRESH`, `SQL_UPDATE` and `SQL_DELETE`. These last three ones belong to the scope "modification". In previous versions, `SQL_ADD` was also allowed for inserting rows, but this option has been placed within `SQLBulkOperations` [33].

Update Calling `SQLSetPos` with a row number and the parameter `SQL_UPDATE` causes the row to be updated in the DBMS based on the values written in the variables linked to the columns [33]. In this way, an application links certain variables to specific columns in the result set, writes the values the application desires into these variables, and sends them to `SQLSetPos` with `SQL_UPDATE` and the row number, which will be the row where one wants to replace the current values associated with that row and those columns with those of the variables associated with those columns, with an effect on the data source.

What MyODBC does is that it constructs an `UPDATE` query in SQL syntax based on the values of the variables and sends it to MySQL Server as any other query.

We observe that we have to replace the call to MySQL with the call to DES. But we also have to be careful with how the query is constructed, since it will not be fully compatible with DES.

DES supports SQL syntax: with the command `/sql`, DES changes the interpreter to a pure SQL interpreter, so that all queries are parsed and executed in SQL. However, some of the queries sent from MyODBC introduced peculiarities related to MySQL.

Our decision was to adhere to the syntax defined by DES's default interpreter for *insert/update/delete* operations, as they are already in SQL format (without needing to use DES-SQL mode) and are therefore very similar to those of MyODBC.

Thus, we would construct the `UPDATE` query based on the following syntax [5, section 4.2.5.3]:

```
UPDATE TableName [[AS] Identifier] SET Att1=Expr1,...,AttN=ExprN
[WHERE Condition]
```

We had to make the pertinent modifications to the process of construction of the query in MyODBC.

Refresh If `SQLSetPos` is called with `SQL_REFRESH`, the application variables associated with the columns are updated based on the current values of the result set, in the row specified by the user [33]. It would therefore be a kind of "inverse of the `UPDATE`" from the previous case. There is no interaction with the DBMS and therefore nothing to modify from MyODBC.

Delete When `SQL_DELETE` is specified in `SQLSetPos`, the row specified by the user is deleted from the data source [33]. Rows in the result set are numbered, but there is no native row indexing in DES¹⁶. Therefore, we had to construct a `DELETE` query based on all the attributes we find in the determined row. We build the *delete* query using the following syntax [5, section 4.2.5.2]:

```
DELETE FROM TableName [[AS] Identifier] [WHERE Condition]
```

However, there is a problem with this syntax: all rows matching the values of the row to be deleted will be removed. MyODBC anticipates this scenario and appends `LIMIT 1` before the sending the query. Certainly, the syntax of modern versions of MySQL allows for the use of the `LIMIT` keyword [34]:

```
DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM tbl_name [[AS] tbl_alias]
  [PARTITION (partition_name [, partition_name] ...)]
  [WHERE where_condition]
  [ORDER BY ...]
  [LIMIT row_count]
```

To overcome this issue, we implement the following algorithm:

1. We find the number of duplicate rows with a statement of the type `select COUNT(*) from table_name WHERE ...`. We store the result in a variable, let's say with a result of n .
2. We remove all duplicate rows using our `DELETE` statement.
3. We insert the deleted row $n - 1$ times.

Special case general to operations: row zero If the row number 0 is specified as an argument, it is defined that the specified operation is executed on all rows of the result set [33]. The only thing MyODBC does is iterate over the number of rows, applying the desired operation one by one. DES also does not offer a more compact way to operate by rows, and therefore we also proceed to perform the operation row by row.

SQLBulkOperations This is a function that, as its name suggests, allows operations to be performed in "bulk". It takes an operation as an argument, which can be `SQL_ADD`, `SQL_UPDATE_BY_BOOKMARK`, `SQL_DELETE_BY_BOOKMARK` or `SQL_FETCH_BY_BOOKMARK` [35].

¹⁶Relational databases in most DBMS are designed to work on datasets that do not have a predefined order.

Add If the `SQL_ADD` flag is indicated, new bulk rows are inserted. For this, you must first specify the number of rows to be inserted by modifying the `SQL_ATTR_ROW_ARRAY_SIZE` attribute of the statement element, then bind the data to be inserted with `SQLBindCol` with an array sized equal to the previously specified number of rows, and finally calling `SQLBulkOperations` with `SQL_ADD`[35].

In any case, everything ultimately translates into constructing an `INSERT` query for each new row to be inserted. As was the case with `DELETE` and `UPDATE`, we have slightly adapted the code to build an `INSERT` query, exactly as expected by DES [5, section 4.2.5.1]:

```
INSERT INTO TableName[(Col1,...,ColN)] VALUES (Expr1,...,ExprN) [,
..., (Expr1,...,ExprN)]
```

Special case general to operations: markers We have seen that the remaining options consist of performing an *update*, *delete*, or *fetch* by bookmark. This translates to the fact that, for each row to which we have added a bookmark in the past, it is updated/deleted/recovered respectively. Again, this has to be achieved row by row. All this is easily implemented based on what has been presented.

3.5.5. Catalog functions (metadata)

Catalog functions, or metadata functions, are a set of API functions that allow us to check certain aspects about the schemas and structure of databases, or some aspects of the DBMS itself.

We consider these functions to be part of this group: `SQLColumns`, `SQLColumnPrivileges`, `SQLGetTypeInfo`, `SQLForeignKeys`, `SQLPrimaryKeys`, `SQLProcedures`, `SQLProcedureColumns`, `SQLSpecialColumns`, `SQLStatistics`, `SQLTables` and lastly `SQLTablePrivileges`.

We will go through them in the following sections based on the category they fall under. Before we begin, we need to explain some preliminaries that will be common to all these catalog functions.

3.5.5.1. Preliminaries

Catalogs, schemas and tables In many of these functions, we are asked to specify, in addition to the name of a table, its associated catalog and schemas.

We introduce the issue of catalogs and schemas at this point. Catalogs, schemas, and relationships form a three-level hierarchy, with catalogs at the top, containing schemas, and each schema defining tables. The schema forms a collection of relations¹⁷ [36, p. 163].

These concepts in practice are not entirely standard: each DBMS interprets them in particular ways. MyODBC finds problematic the distinction catalog / schema: in MySQL, *catalog*, *schema*, and *database* are equivalent concepts in certain situations and are therefore prone to confusion in some contexts. MyODBC allows the user to modify the `NO_CATALOG` and `NO_SCHEMA` options in the data source, so that they are

¹⁷Among the relations, in the catalog functions we will only deal with tables.

aware of the interpretations made by MyODBC as indicated in the documentation (see [37]).

We emphasize that since these concepts are not entirely standard, each DBMS may have its own variations. decision to interpret them according to a standard that they themselves propose in the documentation.

Let's see what happens in DES. First, the types of databases that DES can access are:

- **DES database:** these are databases within the scope of DES. That is, either the default database when starting DES, or one saved as a file. Those saved as files are loaded into DES using the `/restore_ddb` command or `/restore_ddb Filename`, which replaces the current DES database. That is, **only one database can be loaded at a time, bound to a DES session**. Whether it is the default database or a loaded file, when querying the name of the current database, we are informed that it is called `$des`.
- **External databases:** these are databases that we have accessed via an ODBC connection from DES to the ODBC connector of a particular DBMS. That is, we have connected to a database through the `/open_db` command; for example, `/open_db mysql`. In this case, when querying the name of this database, it will be called as the data source to which we have connected; in case we have connected via `/open_db mysql`, the database will be called `mysql`.

We can have loaded into the session both the DES database (always present) and other external databases. Figure 3.9 shows how we can view the list of these databases in DES using the `/show_dbs` command.

```
DES> /show_dbs
$des
access
csv
db2
excel
mysql
oracle
postgresql
sqlserver
```

Figure 3.9: Example output of the call to `/show_dbs` [5, sección 5.1.3].

Nota 1 *It seems confusing to talk about databases in this context. We are calling a "database."^a connection to an external DBMS, which is not entirely accurate: within a connection to an external DBMS, multiple databases can coexist. For example, in MySQL, within the same connection (username-password), you can create and*

use several databases that coexist, as seen in the `CREATE DATABASE` command. Assuming we have created the databases `db1` and `db2` in MySQL under the same user, DES could, within this connection, call `select from bdd1.table` as well as `select from bdd2.other_table`.

*Thus, when we talk about *external databases* in the context of DES, we should actually be referring to *connections*, although these connections then result in us actually working with external databases. We point out this detail to clarify some of these concepts.*

Another aspect to consider is that DES does not have a global overview among the connections. They can only be found within the framework of a specific connection (for example, if we refer to tables from other connections where we are not, I get an error). With `/current_db`, you can find out what connection you are currently in (`$des, access...`) and we can change the connection (i.e., change the context) with `/use_db name`.

With all the details we have gathered from DES, we observe that the concept that fits most naturally with that of the catalog is the concept of connection.

With respect to schemas, DES does not provide support for schemas beyond what is offered by default upon execution of `/dbschema` (or in SQL `DESCRIBE TABLES`, also in DES) across different connections. Therefore, we will do as MyODBC does by applying restrictions to the user: in our case, enforcing that working with schemas is not allowed.

In conclusion, within the metadata functions, we will accept the catalog parameter, which refers to a connection (either `$des` or an external database), the table name, and we will ignore the schema argument.

In some of these metadata functions, specifying an external connection will involve working within the context of that connection: therefore, we will have to use the command `/use_db external_connection` and after finishing the operations restore the previous state by calling `/use_db previous_connection`. The entire process will be carried out with the query lock acquired, in order to *atomize* the operation. From now on, we will omit repeating this implementation detail.

Search patterns In some of these catalog functions, search patterns can be entered in place of full table, schema, or catalog names, a feature introduced in ODBC 3.x. The rules for these patterns can be consulted in the manual (see [38]).

Considering that we only allow the introduction of table names, we could introduce as a name `departamento%`. In this scenario, the function to be called would consider all tables whose name has the `departamento` prefix. To obtain the list of table names to search over, a preliminary call is made to `/dbschema`. For each matching table, we would do the same action that we would have called. The final set of results consists of combining all the rows from the result sets obtained in each of these combinations.

This whole process, of course, with the query lock acquired to *atomize* this

operation.

From now on, we will only discuss introducing table names, without referring to this search possibility (in some functions, this is allowed and in other cases, no, but indicating where is irrelevant for this document).

3.5.5.2. Columnas

SQLColumns The list of columns is returned given search patterns for a table name and/or column name [39]. To do this, a call is made within the specified connection to `select * from table_name` to access the column names of the table. If column names are specified in the parameters, non-matching columns are filtered from the result set.

Internally, DESODBC extracts information from the output of this *select* query and constructs the expected result set according to the **SQLColumns** specification. It is necessary to call `select * from table_name` instead of a simple `/dbschema connection_name:table_name` since the query results return information related to the data, such as the maximum number of significant decimal places that appear in the column's data.

SQLColumnPrivileges This function takes the name of a table as a parameter and returns the privileges associated with the columns of that table and returns information about the privileges for each of the columns [40]. The concept of privileges does not exist in DES, and therefore we mark the function against the user as "not implemented".

Nota 2 *One might wonder what happens if, in this function, the request for privileges is made within a catalog that refer to a connection to a DBMS that supports privileges. In this case, from DES's perspective, they do not exist: there are no DES commands to access these types of features in a connection that other DBMS implement, such as privileges, procedures, indices... DES allows knowledge of other connections similar to what could be known from a \$des database¹⁸. Therefore, the client should not interpret a connection from DES to another DBMS as a regular connection to be the real", but rather that DES accesses the data itself from this database, and interprets and structures it according to DES's way of proceeding.*

*Therefore, when an external connection has been specified and the ODBC function potentially allows returning such external elements as part of the result set, a **SQL_SUCCESS_WITH_INFO** message will be issued indicating that information about indexes, procedures, page count, or privileges may have been omitted. In non-implemented*

¹⁸We can send external connections queries typical of these DBMS so that they read and manipulate these types of elements. However, although we can resort to this possibility, we are referring to the fact that DES does not have its own commands or features to access "native characteristics that are not present in DES databases. Since each DBMS will have its own set of queries to access these elements, we opt to restrict by design the consultation of the same.

functions like *SQLColumnPrivileges*, if an external connection is specified, the corresponding information will be included in the error message.

After explaining this issue, we will omit reiterating it when discussing other functionalities that DES also does not implement, such as procedures and indexes.

SQLSpecialColumns *SQLSpecialColumns* allows to know for a table the set of "specialcolumns, understanding by special those that uniquely identify the rows of the table (primary keys), and those that are automatically updated when a transaction that has modified some value of these columns is performed [41]. DES does not support transactions and therefore we only return the primary keys. This process will be virtually identical to the one carried out in *SQLPrimaryKeys*, and therefore we will comment on how to obtain the primary keys in said section.

3.5.5.3. Claves

The functions *SQLPrimaryKeys* and *SQLForeignKeys* allow which columns of a table are associated with primary and foreign keys, respectively.

The command `/dbschema nombre` allows us to know, given a table name, among other things, its primary and foreign keys. In Figure 3.10, an example of the output of this command can be seen. By analyzing this output is how we will be able to implement both functions.

```
DES> /dbschema registration

Info: Database '$des'
Info: Table:
* registration(student:varchar(45),course:varchar(11),pass:int)
  - NN: [student,course]
  - PK: [student,course]
  - FK: registration.[course] -> courses.[id]
  - FK: registration.[pass] -> prueba.[id]
```

Figure 3.10: View of the primary and foreign keys of a table in DES.

We note that it is not possible to determine the primary and foreign keys of tables from external connections (those not underlined within the *dbschema*). This is likely due to the inability to determine them with a procedure that works for all DBMS systems. When the user specifies a connection different from *\$des* in a catalog field, this limitation will be communicated.

With *SQLPrimaryKeys*, it is expected that, given a table name, it returns the column names of the primary keys, properly formatted [42]. The implementation of *SQLForeignKeys* is more complex as it supports the following possibilities as arguments [43]:

1. Introduce a table in a field `PKTableName`.

Return the primary keys of said table and, for each primary key, foreign keys that refer to it.

2. Introduce a table in a field `FKTableName`.

Foreign keys in a table that reference primary keys in other tables are returned.

3. Introduce a table with two fields.

Merge of both cases: the foreign keys are returned from the table named `FKTableName` that reference the primary keys of the table named `PKTableName`.

We emphasize these details since they require a more delicate implementation. In particular, we will need to know the primary and foreign keys of all tables; this can be achieved by calling `/dbschema`, as if no table is specified, it displays the schema of all relationships. Therefore, we first extract the information from the TAPI format output of `/dbschema` and introduce it into the appropriate data structures of the connector (see listing 3.19), and then we will consult the same information according to the needs specified in the arguments¹⁹.

```

1 // driver/driver.h (DESODBC)
2 struct ForeignKeyInfo {
3     std::string key;
4     std::string foreign_table;
5     std::string foreign_key;
6 };
7
8 struct DBSchemaTableInfo {
9     // Unordered maps name -> column index
10    std::unordered_map<std::string, int> columns_index_map;
11    std::unordered_map<std::string, Type> columns_type_map;
12    std::vector<std::string> primary_keys;
13    std::vector<ForeignKeyInfo> foreign_keys;
14    std::vector<std::string> not_nulls;
15
16    std::string name;
17 };

```

Listing 3.19: Structures storing information from `dbschema`.

3.5.5.4. Procedures

The standard specifies that `SQLProcedures` returns the list of procedures stored in a data source"[44]. It is mentioned in this reference that procedures, in the abstract, are executable procedures (for example functions that take some arguments

¹⁹We atomize this operation by acquiring the query lock for the entire process, so that the data remains consistent. We proceed similarly in other situations where there is a risk of losing consistency, but we will omit mentioning it.

and return an output). Generally, they are SQL statements. The benefit of these functions is that they allow you to refer to an entire procedure by using an alias, thus avoiding repeating code. In Figure 3.11, you can see an example of a procedure in MySQL.

Procedures as understood in the SQL standard are not present in DES (nor in `/sql` mode). There is a certain semantic similarity between the concept of *procedure* as an executable object and the Datalog rules that can be established in a DES database. Technically, we could implement **SQLProcedures** based on this convention. We chose not to do so for the following reasons:

```
mysql> CREATE PROCEDURE citycount (IN country CHAR(3), OUT cities
                                INT)
    BEGIN
        SELECT COUNT(*) INTO cities FROM world.city
        WHERE CountryCode = country;
    END
Query OK, 0 rows affected (0.01 sec)

mysql> CALL citycount('JPN', @cities); -- cities in Japan
Query OK, 1 row affected (0.00 sec)
```

Figure 3.11: Example of procedures in MySQL. Extracted from [45].

1. Ultimately, **SQLProcedures** is defined within the strict context of SQL, as indicated by its very prefix. The user expects to obtain "procedures" within the scope of the SQL standard, which deviate completely from the specific syntax and features characteristic of Datalog rules.
2. Datalog rules define predicates in a special-purpose language, not a general one; therefore, they are not referred to as "*procedures*" or procedures in any standard.

DES also presents a Turing-complete scripting language through which we can approach the concept of procedures. However, we also discard it for the same reason of lack of strict adherence to the SQL standard regarding what is understood by *procedures*.

We chose to mark this function as "not implemented," along with **SQLProcedureColumns**, another function that also operates on procedures.

3.5.5.5. Tables

We refer to the functions **SQLTablePrivileges** and **SQLTables**. The first function is marked as unimplemented; we have established that the concept of privileges does not exist in DES. **SQLTables** simply returns the list of tables according to search patterns placed in the catalog, schema, table, and relationship type (table or view) arguments [46].

3.5.5.6. Types

The ODBC standard distinguishes two types of data types: SQL data types and C data types.

The SQL data types are those associated with the data types supported by the DBMS that runs SQL. In other words, standard SQL defines a set of data types that are widely supported across different database management systems, which afterwards each DBMS is responsible for mimicking among all that SQL offers.

On the other hand, C data types are used in the application to specify data types for application variables. They are simply data types used in the application's C code, written in C/C++. Therefore, these data types are defined by C specification.

This is in line with the buffer in terms of the standard agreed upon between the application and the connector; C data types are types agreed between both ends (this is possible since both the application and the connector know C/C++).

The "agreement" stems from the fact that both know the ODBC standard, which establishes a correspondence between the alias of C data type and the actual C data type. See examples in Table 3.2.

C type identifier	ODBC C typedef	C type
SQL_C_CHAR	SQLCHAR *	<code>unsigned char *</code>
SQL_C_FLOAT	SQLREAL	<code>float</code>
SQL_C_BIT	SQLCHAR	<code>unsigned char</code>

(*ODBC C typedef* corresponds to a C-style alias for a type —similar to `using`—, to avoid dealing with it directly).

Table 3.2: Some examples of correspondence between C data types and C ODBC data types", extracted from [47].

For example, if an application or driver encounters a buffer that has been assigned a data type of numeric identifier `SQL_C_CHAR` (defined under certain numbers in the ODBC and SQL header files), you know that you have to treat the information from such buffer as `SQLCHAR *`, an alias for `unsigned char *`.

The standard defines a correspondence between SQL data types and the data types in C, which the connector should handle the implementation of. Consider the following complete example:

An application has executed a query. It encounters that the cursor is positioned in a specific row. He wants to know the value that exists within that row in a determined column. Therefore, he calls the `SQLGetData` function, whose header can be observed in listing 3.20.

```

1 SQLRETURN SQLGetData(
2     SQLHSTMT StatementHandle,
3     SQLUSMALLINT Col_or_Param_Num,
```

```

4     SQLSMALLINT TargetType,
5     SQLPOINTER TargetValuePtr,
6     SQLLEN BufferLength,
7     SQLLEN * StrLen_or_IndPtr);

```

Listing 3.20: ODBC API function `SQLGetData` header [48].

In `TargetType`, the application specifies what type of data C expects to find.

Let's assume it wants to retrieve the value in a date-type column. By making a series of prior calls, it discovered that the internal type of this column is `SQL_TYPE_DATE`.

The ODBC standard states that an `SQL_TYPE_DATE` can be converted to both `SQL_C_CHAR` and `SQL_C_TIMESTAMP` [49], with the corresponding data structure shown in Listing 3.21.

```

1 struct tagTIMESTAMP_STRUCT {
2     SQLSMALLINT year;
3     SQLUSMALLINT month;
4     SQLUSMALLINT day;
5     SQLUSMALLINT hour;
6     SQLUSMALLINT minute;
7     SQLUSMALLINT second;
8     SQLINTEGER fraction;
9 } TIMESTAMP_STRUCT;

```

Listing 3.21: Structure associated with `SQL_C_TIMESTAMP`. Extracted from [47].

Examples of transformations from SQL type to C type can be seen in Table 3.3.

<i>SQL type id.</i>	<i>SQL data value</i>	<i>C type id.</i>	<i>Buf. length</i>	<i>*TargetValuePtr</i>
SQL_TYPE_DATE	1992-12-31	SQL_C_CHAR	11	1992-12-31\0
SQL_TYPE_DATE	1992-12-31	SQL_C_TIMESTAMP	ignored	1992, 12, 31 , 0, 0, 0, 0

Table 3.3: Some examples of concrete transformations between types SQL-C, extracted from [49].

The application eventually wants to treat this date as a string, so it introduces the value in `TargetType` the value `SQL_C_CHAR`, so according to Table 3.2 it will obtain a value of type `SQLCHAR*`, alias of `unsigned char*`; and in `TargetValuePtr` it places the pointer to a buffer of type `SQLCHAR*`.

When the function is called, the connector internally performs a cast from a date value from the DBMS to a string value, and finally writes this result into the buffer pointer passed as an argument. Listing 3.22 shows the concrete example.

```

1  ok_sql(hstmt, "create or replace table test(d date)");
2  ok_sql(hstmt, "insert into test values(cast(\'2025-02-13\' as date))");
3  ok_sql(hstmt, "select * from test");
4
5  ok_stmt(hstmt, SQLFetch(hstmt));
6
7  SQLCHAR datebuf[TEST_BUFFER_SIZE];
8  SQLLEN len;
9
10 ok_stmt(hstmt, SQLGetData(hstmt, 1, SQL_C_CHAR, datebuf, sizeof(datebuf),
11                             &len));
12
13 is_str("2025-02-13", datebuf, len);
14
15 return OK;

```

Listing 3.22: Test application (see Section 3.6) that checks the correct casting to the real type of C through calls to DESODBC. First, it inserts a date type value into DES and then retrieves it as a string type.

Let's discuss what happens in our connector. Remember that transformations between C data types and SQL data types must be done from the driver: Fortunately, we can reuse all the MyODBC code in this regard. However, we had to redo the MyODBC aspect related to MySQL types, as they differ from DES types.

Let's start by considering the correspondence between the data types defined in DES to SQL data types, considering the valid data types in tuples as observed in Table 3.4.

<i>Type</i>	<i>Meaning</i>
<i>varchar, string</i>	<i>String of unbounded length</i>
<i>char(N), varchar(N)</i>	<i>String with length up to N</i>
<i>char</i>	<i>String with length 1</i>
<i>integer, int</i>	<i>Integer number</i>
<i>float, real</i>	<i>Real number</i>
<i>date</i>	<i>Date expressed as <code>date(Year, Month, Day)</code></i>
<i>time</i>	<i>Time expressed as <code>time(Hour, Minute, Second)</code></i>
<i>datetime, timestamp</i>	<i>Timestamp expressed as <code>datetime(Year, Month, Day, Hour, Minute, Second)</code></i>

"Precision and range depend on the underlying Prolog system."

Table 3.4: DES data types [5, section 4.1.16.1]²⁰.

²⁰EThe manual also lists in [5, sección 4.2.4.1] data types that can be specified during table creation, including NUMERIC and DECIMAL (which accept parameters). These are internally translated

Based on this last direct quote from the manual, we need to see what precisions and ranges are established in the two possible Prolog systems on which there are versions for DES: **SICStus Prolog** or **SWI-Prolog**.

SICStus Prolog considers [50, sección 4.7.1]:

1. The range of integers is unbounded.
2. Floating-point types occupy 64 bits and their ranges are correspond to the `double` type in C, typically in the range `[4.9e-324, 1.8e+308]`.

SWI-Prolog subscribes to the same considerations. However, the range of integers is unlimited only when **SWI-Prolog** is compiled with *GNU multiple precision arithmetic library (GMP)*; if not, integers are 64-bit [51, sección 4.27]. In the prototypical case using the already compiled version of DES with **SWI-Prolog**, we confirm that the range of integers is unlimited.

In any case, it is a problem to consider integers of unbounded range: such a thing is not natively found in C, and therefore, there is no associated C data type. In the latest version of ODBC, ODBC 3.8, it allows defining specific C types in the driver[52]: however, in this process the range of the type must be bounded between a series of values.

An implementation of unlimited integers that overcomes this problem could be material for future work. Therefore, for now we will consider that integers are 64-bit. In this way, we will also be conservative with the residual case where the client uses a DES compiled with a version of **SWI-Prolog** compiled without the GMP library.

The final correspondence of DES types to SQL types is presented in Table 3.5.

3.5.5.7. Statistics

We refer to the **SQLStatistics** function, which is expected to return a series of features of a given table when provided with a table name [54]. Most of these features relate to indexes, which are not supported by DES; however, only table information can be returned by specifying that a result set of type **SQL_TABLE_STAT** is returned. In this case, the following will be returned:

1. **CARDINALITY**: in our case, the number of rows in the table.
2. **PAGES**: in our case, the number of pages used to store the table.

This last value is optional and in fact we will not return it, since we cannot know this value through simple requests to DES.

to the types shown in the table. The connector only deals with this latter set of types, as queries about data types from the connector only occur for already-created tables (and thus, their types have already been translated).

DES data type	Id. of SQL typeSQL	SQL data type	Precision (bits)	Range
varchar / string	SQL_LONGVARCHAR	LONG VARCHAR	—	—
char(<i>N</i>)	SQL_CHAR	CHAR(<i>N</i>)	—	—
varchar(<i>N</i>)	SQL_VARCHAR	VARCHAR(<i>N</i>)	—	—
char	SQL_CHAR (*)	CHAR(1)	—	—
integer / int	SQL_BIGINT	BIGINT	19 (***)	$[-2^{63}, 2^{63} - 1]$
float / real	SQL_DOUBLE (**)	DOUBLE PRECISION	53	$[10^{-308}, 10^{308}]$ (in abs.)
date	SQL_TYPE_DATE	DATE	—	—
time	SQL_TYPE_TIME	TIME(<i>p</i>) (****)	—	—
datetime / timestamp	SQL_TYPE_TIMESTAMP	TIMESTAMP(<i>p</i>)	—	—

(*) In SQL, there is no concept of *char* as it is understood in C or DES as "unique character". It will then be implemented from the perspective of a string of length 1.

(**) In DES, a *float* is synonymous with *real*, which is not the case in SQL.

The *float* and *real* types in DES refer to the SQL DOUBLE data type.

(***) It would be 20 if we allowed a *double* of type *unsigned*, but it does not fit here. In DES there are no unsigned types.

(****) In the case of TIME and TIMESTAMP, the dependent parameter *p* is considered, which indicates the second's precision. The SQL standard itself considers this parameter to be optional, and it makes no sense in DES, so we will ignore it.

Table 3.5: Correspondence of DES data types with SQL data types. Precisions and ranges can be consulted in [53].

3.5.6. Error management system

3.5.6.1. Driver errors

Quality software should implement sufficiently comprehensive error handling. Fortunately, ODBC helps us with this task. ODBC API functions return a value of type `SQLRETURN`. This type includes the self-explanatory values `SQL_SUCCESS`, `SQL_SUCCESS_WITH_INFO`, `SQL_ERROR`, and so on.

When the user executes a function that obtains values carrying information, such as `SQL_SUCCESS_WITH_INFO` or `SQL_ERROR`, if they want to know the associated message, they must call the ODBC API function `SQLGetDiagRec`. This function returns a pointer to the string associated with the diagnostic field of the specified handler [55].

Therefore, in the implementations of the functions, whenever we are in a situation

that could be reported to the user, we must fill in the diagnostic field of the handler we are working with (connection, statement, etc.). This allows us to inform the user of errors when they occur.

Let's see an example of error handling. The code in Listing 3.23 handles the creation of the output pipe in Windows, within a function `DBC::create_pipes`.

```

1 // driver/connect.cc (DESODBC)
2 SQLRETURN DBC::create_pipes() {
3     ...
4     if (!CreatePipe(&this->driver_to_des_out_rpipe,
5                     &this->driver_to_des_out_wpipe, &des_pipe_sec_attr,
6                     des_pipe_buf_size)) {
7         CloseHandle(this->driver_to_des_out_rpipe);
8         CloseHandle(this->driver_to_des_out_wpipe);
9         return this->set_win_error("Failed to create DES output pipe", true);
10    }
11    ...

```

Listing 3.23: Error management when creating the output pipe in Windows, within a `DBC::create_pipes` function.

When calling `CreatePipe` (`namedpipeapi.h`), if we obtain `false` it indicates an error from this function.

Therefore, we first manage this by safely closing the pipe handles, and then calling a function from the DBC called `set_win_error`. This function is designed for errors specifically related to Windows: its task is to write in the diagnostic field of the DBC the message specified in one of its arguments, and additionally include in that message the diagnostic information specific to Windows (using `FormatMessage` — `winbase.h` — on `GetLastError` — `errhandlingapi.h`). There is an analogous function for Unix-based systems called `set_unix_error`; and another series of generic error functions (not related to operating system errors, but to all others) called `set_error`.

In any case, the `set_error` type function also returns the corresponding `SQLRETURN` value: in this particular case, `SQL_ERROR`. Therefore, the flow of `DBC::createPipes` stops at this point, and the error is caught by the function that called it. The error in this case is received by a function `DBC::connect`, immediately stops its flow, and returns the error to the calling function in the stack, the ODBC API function `SQLConnect`. This function captures the error and immediately returns it, so globally, the user has obtained an `SQL_ERROR` by calling `SQLConnect`. If it wants to know the diagnostic message, we insist, it must subsequently call `SQLGetDiagRec`.

3.5.6.2. Errors in DES

An ODBC connector also has the task of capturing and managing errors generated in the target DBMS. So, if in an ODBC API function, such as `SQLExecDirect`, we execute a query related to a non-existent table, our DBMS will notify us. It is then

our responsibility to capture this error, return the appropriate `SQL_SUCCESS_WITH_INFO` and `SQL_ERROR` status codes.

Fortunately, DES provides a general error format for commands in the TAPI, which we can use to observe DES's own diagnosis after a query (see Figure 3.12).

<code>\$error</code>
<code>code</code>
<code>text</code>
<code>...</code>
<code>text</code>
<code>\$eot</code>

Figure 3.12: Generic error message in TAPI [5, section 5.18.1.2].

The `code` field can take on the values 0 (denoting an exception error), 1 (*warning*), or 2 (informational message) [5, section 5.18.1.2]. We observe that we can map 0 to `SQL_ERROR`, and 1/2 to `SQL_SUCCESS_WITH_INFO`.

3.5.7. ANSI and Unicode versions of the driver

MyODBC offers its ODBC connector in two versions: ANSI and Unicode.

MySQL understands Unicode as a coding that supports wide characters (UTF-16). However, using the term *Unicode*,^{as} MyODBC does is not entirely correct, the ANSI standard also supports Unicode, but specifically UTF-8.

Both UTF-8 and UTF-16 can encode any Unicode character. The thing that happens is that UTF-8 uses 8-bit "blocks" to do so, and UTF-16 uses 16-bit blocks. For instance, let's see how a particular Chinese character U+24B62 is represented in the following encodings in Table 3.6.

UTF-8 encoding	0xF0 0xA4 0xAD 0xA2
UTF-16 encoding	0xD852 0xDF62
UTF-32 encoding	0x00024B62

Table 3.6: Different Unicode encodings of the Chinese character in question. Extracted from [56].

It is for this reason that, even though they may represent the same number of characters, UTF-16 is incompatible with UTF-8 and that is why MyODBC differentiates between these two versions.

Performing pertinent tests, DES does not support UTF-16 (but does support UTF-8). Furthermore, it only processes the alphabet of Western European languages: ñ, ö, á, à. We conclude that DES supports the character set known as Latin1

(ISO/IEC 8859-1). Fortunately, MyODBC allows you to select charsets from a large list stored in a MySQL folder, which includes `Latin1`.

We find ourselves in the predicament of whether to eliminate the Unicode version or not. Essentially, the functions of the ODBC API with the suffix `W` (referring to those that use wide characters, or in the terminology of MyODBC, `Unicode`), in the MyODBC implementation it never uses the wide character type `wchar_t`; the connector is always limited to previously converting them to simple string characters `char`²¹. Let's see the example presented in the code listing 3.24.

```

1 // driver/unicode.cc (DESODBC)
2 SQLRETURN SQL_API
3 SQLPrepareWImpl(SQLHSTMT hstmt, SQLWCHAR *str, SQLINTEGER str_len,
4                 bool force_prepare)
5 {
6     STMT *stmt= (STMT *)hstmt;
7     uint errors;
8     SQLCHAR *conv= sqlwchar_as_sqlchar(stmt->dbc->cxn_charset_info,
9                                       str, &str_len, &errors);
10    /* Character conversion problems are not tolerated. */
11    if (errors)
12    {
13        x_free(conv);
14        return stmt->set_error("22018", NULL);
15    }
16
17    SQLRETURN rc = DESPrepare(hstmt, conv, str_len, false, force_prepare);
18    x_free(conv);
19    return rc;
20 }

```

Listing 3.24: Example of translation double-spaced characters to strings of type `char`.

Therefore, considering the function `sqlwchar_as_sqlchar`, we just need to ensure that `stmt->dbc->cxn_charset_info` refers to the charset `Latin1`.

The reason for having a Unicode version for DES would then be limited to providing the user with the ability to use strings with double-byte character, which, although it may seem unnecessary since it cannot exploit the full Unicode repertoire in DES, might be the most convenient for certain coding circumstances within your application. Fortunately, we won't have to modify anything in the code.

²¹This does not imply any loss of information. While `char` is designed to store ASCII-type characters (8 bits instead of 16), internally there has been no loss of information; the string will simply occupy double the space. The only detail is that if we read it as a `char` string, it will represent a nonsensical string. We will have to reinterpret the internal information, which has not been lost, as a string of `wchar`, when we want to interpret it.

3.6. *Testing*

MyODBC provides a large number of tests in its source code. The tests consist of 35 source files, each focused on a specific aspect of the connector, with multiple unit tests, averaging around a thousand lines of code per file, and designed specifically for MySQL. Therefore, we were able to reuse very few of them. We proceeded to eliminate almost all the tests and reuse the testing framework employed by MyODBC, defined in a header file `odbctap.h`.

When a source file includes `odbctap.h`, a connection to the data source is automatically established, and thus the tests should only include the specific operations to be verified, simplifying the testing process. Listing 3.25 shows an example of a test.

Thus, we have generated a total of 20 tests, which have been useful for debugging and ensuring that the most important potential bugs do not occur. The final list of tests includes:

- Sending a simple `SELECT` query and correctly retrieving the entire set of results using the relevant ODBC API functions.
 - For a standard cursor (scrollable static cursor).
 - For a block cursor (block cursor).
- Parameter binding.
- Use of application variables.
- Type conversion.
- Execution of `SQLColumns`.
- Execution of `SQLGetTypeInfo`.
- Execution of metadata functions over primary (`SQLPrimaryKeys`) and foreign keys (`SQLForeignKeys`).
- Execution of `SQLSpecialColumns`.
- Execution of `SQLStatistics`.
- Execution of `SQLTables`.
- Execution of operations `INSERT`, `REFRESH`, `DELETE`, `UPDATE` (`SQLSetPos`).
 - For a standard cursor (scrollable static cursor).
 - For a block cursor.
- Execution of operations on bookmarks.
- Execution of bulk operations.

- Disconnection using `SQLDisconnect`.
- Proper resource release with `SQLFreeHandle`.
- Error handling check.
- Obtaining attributes of the connector and statement and connection elements.

```

1 #include "odbctap.h"
2
3 DECLARE_TEST(basics)
4 {
5     SQLLEN nRowCount;
6
7     ok_sql(hstmt, "DROP TABLE IF EXISTS t_basic");
8     ok_sql(hstmt,
9         "CREATE TABLE t_basic (id INT PRIMARY KEY, name VARCHAR(20))");
10    /* Insert 3 rows of data */
11    ok_sql(hstmt, "INSERT INTO t_basic VALUES (1,'foo'),(2,'bar'),(3,'baz')");
12    ;
13    /* SELECT query */
14    ok_sql(hstmt, "SELECT * FROM t_basic");
15    /* Check if the number of cols is 3 */
16    ok_stmt(hstmt, SQLNumResultCols(hstmt, &nRowCount));
17    is_num(nRowCount, 3);
18    ...
19    return OK;
20 }
21
22 BEGIN_TESTS
23     ADD_TEST(basics)
24     ...
25 END_TESTS
26
27 RUN_TESTS

```

Listing 3.25: Structure of a test file. Reused from MyODBC and modified for the document.

Each of these tests checks both that the expected postconditions are obtained and the absence of memory leaks. They are designed to verify the correct operation of all implemented ODBC API library functions (although not too exhaustively, given the vast number of possible scenarios that can occur using the driver).

3.7. Documentation

Proper documentation is an essential aspect of quality software. The MyODBC documentation consists of a few online resources primarily focused on the installation

process²² and comments in the headers of implemented functions.

In our case, this document itself serves as a form of documentation, while we comment on the most important connector functions: we retain those of MyODBC that remain valid (and reference their authorship, respecting their rights under the GPLv2 license), and we write new comments for the new, most important implemented functions and symbols. An example of function documentation can be seen in Listing 3.26.

```
1  /* DESODBC:
2     Removed the unnecessary fields of MyODBC's setup library
3     regarding DESODBC's needs and introduced the new ones.
4     Original author: MyODBC
5     Modified by: DESODBC Developer
6  */
7  // OnWMNotify - provides the tooltip control with the appropriate text
8  ...
9  VOID OnWMNotify(WPARAM wParam, LPARAM lParam)
10 {
11     LPTOOLTIPTEXT lpttt;
12     int idCtrl;
```

Listing 3.26: Example of documentation of a function.

3.8. On development and debugging: software used

Given that the development strategy consisted of fully implementing Windows and then exporting it to Unix systems by modifying a few aspects such as IPC, most of the development and debugging work was carried out on Windows. We now proceed to describe the software used on both systems.

3.8.1. Independent of the system

Git We used Git, on Windows through the GitHub Desktop utility. Thanks to it, we were able to maintain a clean version control of the entire DESODBC suite in a GitHub repository.

GIMP GIMP is an open-source, cross-platform image editing software. We used this application to redesign the MySQL-related banner in the configuration library, adapting it to DES (see Figure 3.13).

3.8.2. Windows

The Windows system used during development was Microsoft Windows 10 22H2.

²²We refer primarily to [57].

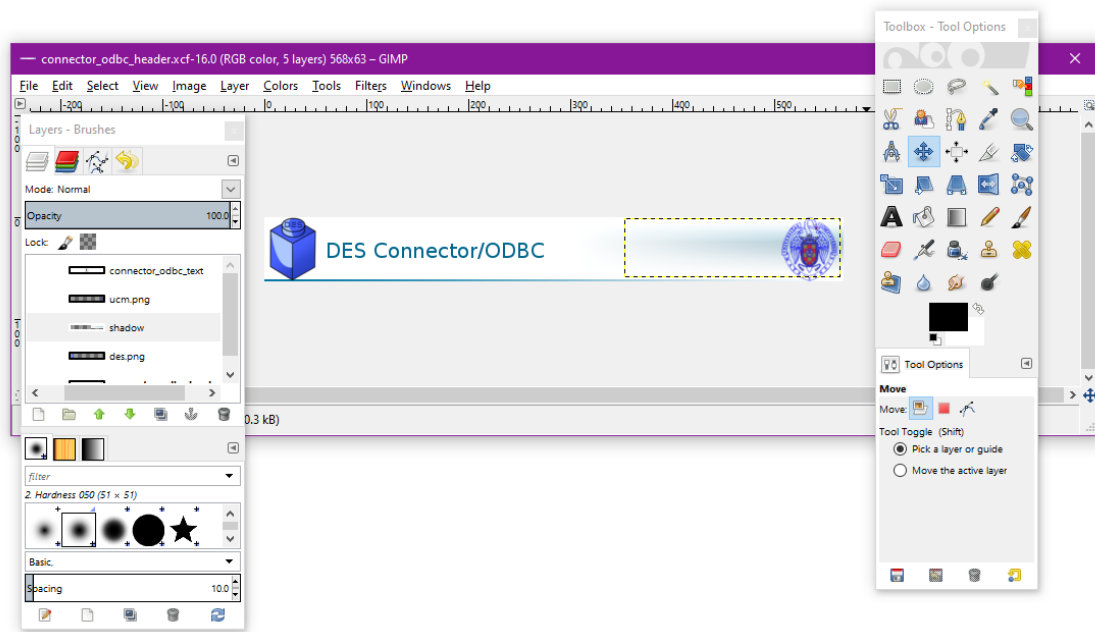


Figure 3.13: Redesigning the *banner* with GIMP.

The main development tool for the connector was Visual Studio Community 2022. MyODBC provided a `.sln` file, a Visual Studio solution file, which encompassed all the necessary components. projects we have modified: ANSI and Unicode versions of the connector, configuration library, installer, tests, and some utilities. Therefore, the development of the entire DESODBC suite was primarily carried out from Visual Studio.

3.8.2.1. Functionalities of Visual Studio

Some specific features of Visual Studio that we used and are worth mentioning for their great utility were the following:

Local Windows Debugger command When debugging from Visual Studio, there is the possibility to run an initial command. This command is specified, starting from the desired project to debug, in *Configuration Properties* → *Debugging* → *Command*. The command we have used is to enter the path of an executable, so that it starts. When the executable interacts with our libraries compiled in debug mode, the necessary breakpoints in the driver code are activated. The main executable we used for these purposes was *ODBC Test*, which we will discuss now.

Linking to a process Sometimes it was not possible to debug from the command due to the nature of executables: for instance, command-line executables. This was a problem when attempting to run test programs that interacted with our connector.

This issue was resolved by manually running the desired executable, and within the Visual Studio connector project, link the connector to the executable using the *Attach to Process* functionality. From there, you can search for our executable among

the list of active PIDs and link it to the same. From there, you can debug using *breakpoints* and derivatives, as described earlier.

Dialog Editor The *Resource Editors*²³ are Visual Studio tools that allow to edit with the comfort of a GUI the *resource script* (.rc) files as the one that defines the GUI of our configuration library, which in this case is a Dialog and therefore we use the *Dialog Editor* tool²⁴. In Figure 3.14, we can see the use of this tool to design the graphical interface of the configuration library.

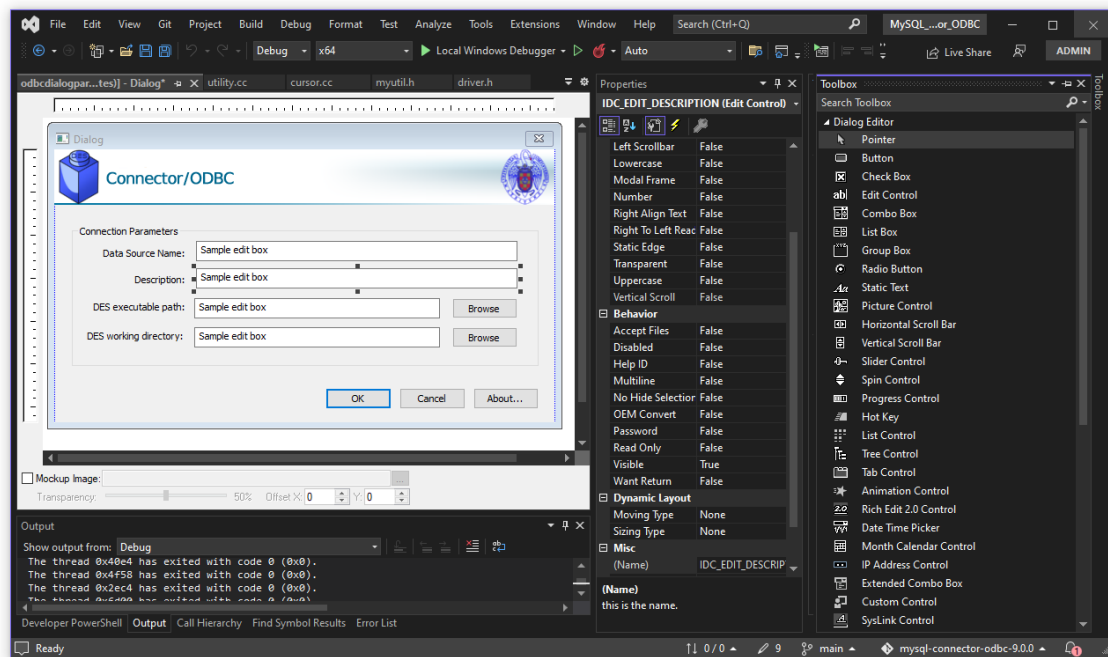


Figure 3.14: Using the *Dialog Editor* for the configuration library.

3.8.2.2. Software unrelated to Visual Studio

ODBC Test The primary software we have used for debugging is ODBC Test 02.70²⁵, a simple graphical interface application that allows, among other things, to execute ODBC API functions over a specified connector (see Figure 3.15).

ODBC Data Source Administrator Introduced in Figure 3.2, this software provided us with a graphical interface for adding, deleting, and modifying data sources and ODBC drivers.

Registry Editor The *Registry Editor* software is built into Windows and provides a graphical interface for easily modifying Windows registry entries. This allows us

²³<https://learn.microsoft.com/en-us/cpp/windows/resource-editors?view=msvc-170>

²⁴<https://learn.microsoft.com/en-us/cpp/windows/dialog-editor?view=msvc-170>

²⁵<https://learn.microsoft.com/en-us/sql/odbc/odbc-test?view=sql-server-ver16>

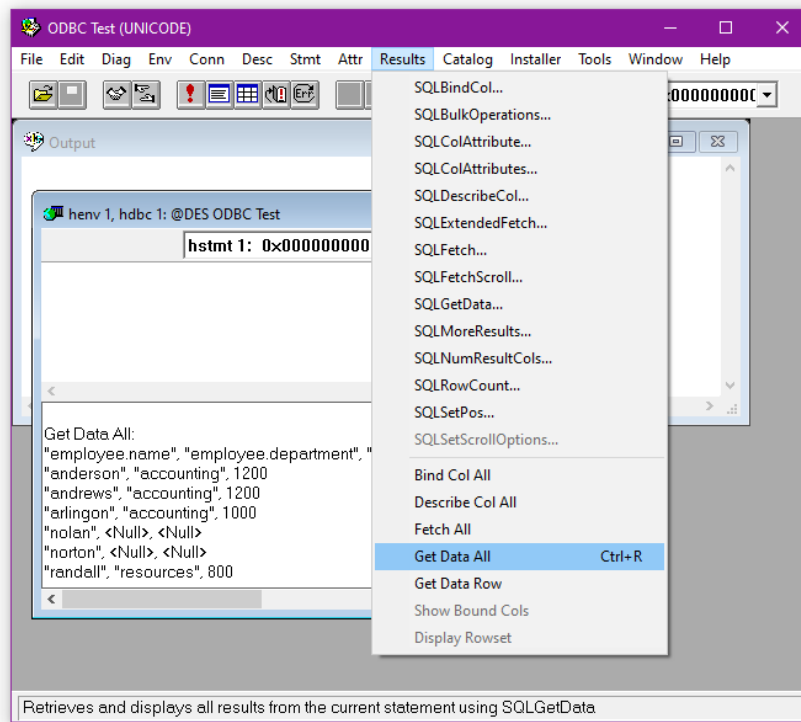


Figure 3.15: Using the "Get Data All" functionality in ODBC Test, which after the execution of a command, calls the sequence of relevant API functions to display the formatted set of results.

to directly access the `odbc.ini` and `odbcinst.ini` files without going through the ODBC Data Source Administrator, so that among other things we can add new key-value entries that are not covered by the configuration library interface, allowing us to perform certain tests.

Dependency Walker In the process of compiling MyODBC during the initial development steps, you need to compile the `mysqlclient.lib` library in Debug mode and deal with dependency issues with other libraries, we had to resort to the Dependency Walker 2.2 software²⁶ to study the dependency tree that existed between certain libraries.

AccessChk AccessChk²⁷ is a tool from the *Sysinternals* suite that allows to the operating system's internal resources. We had to make use this utility (v6.15) to see in real time what IPC files (shared memory, pipes) existed in memory. For example, to display the live memory resources that match partially or fully by name with `DESODBC_SHMEM`, it is executed in a *shell*:

```
accesschk.exe -osv | findstr DESODBC_SHMEM
```

²⁶<https://www.dependencywalker.com/>

²⁷<https://learn.microsoft.com/es-es/sysinternals/downloads/accesschk>

This allowed us not only to debug how our connector handled these resources, but also to understand how Windows manages them, as we explained in the section on disconnection.

Process Explorer This software was already introduced in section 3.5.2.2, when we discussed the issue of pipes with the DES process. We used version v17.06. It allowed us to verify that it was not feasible to access the input and output handlers that the DES process exposes externally.

Event Viewer *Event Viewer* is an integrated Windows software that allows observation of vents launched by applications. By events, we do not mean the event objects used to describe IPC processes in Windows, but rather specific internal messages generated through the Windows API functions `RegisterEventSource` and `ReportEvent` (`winbase.h`). With the Event Viewer, we can see from a global perspective which messages have been sent, from which applications and when. This was necessary when we could not debug with the other tools mentioned on account of several reasons.

Shell utilities PowerShell shell was used to execute multiple system utilities in our development. One of them, for example, was to check the status of processes by their PID (for instance, `Get-Process -Id 11332`). This allowed us to know when and how DES processes were destroyed in the disconnection processes.

3.8.3. Unix

Unix-based operating systems used for development were Ubuntu 22.04.5 LTS (Jammy Jellyfish) and macOS Big Sur 11.0.1.

Visual Studio Code Visual Studio Code is a free, open-source code editor developed by Microsoft. We have used it to work on the source code of our connector and configuration library on GNU/Linux and macOS.

Glade (GNU/Linux) As we mentioned in the section on developing the configuration library, the software Glade²⁸ allowed us to design the configuration library for GTK versions. We used different versions of Glade 3.8.0 for GTK-2 and Glade 3.22.2 to perform the GTK-3 version due to compatibility issues. As shown in Figure 3.16, this tool is employed to design the graphical interface for the configuration libraries.

unixodbc-gui-qt (GNU/Linux) This software²⁹ comprises a suite of auxiliary tools to unixODBC that provide a Qt-based graphical interface. This allowed us to use the tools `ODBCCreateDataSourceQ5` and `ODBCManageDataSourcesQ5` to create, modify, and delete data sources and connectors comfortably.

²⁸<https://glade.gnome.org/>

²⁹<https://github.com/mazbrili/unixodbc-gui-qt>

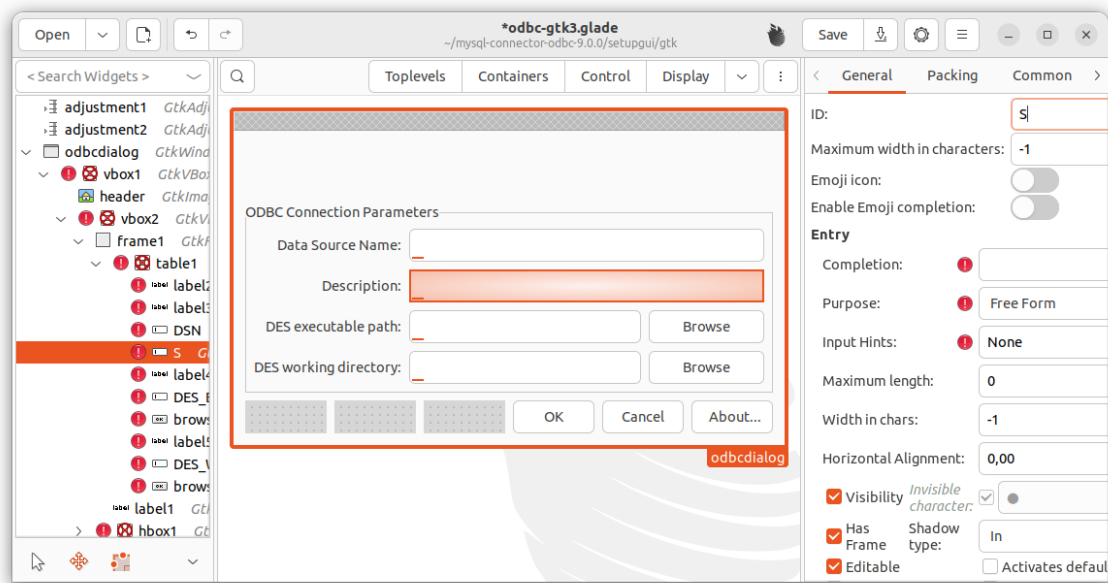


Figure 3.16: Glade 3.22.2 for developing our configuration library for GTK-3.

iODBC Data Source Administrator (macOS) It is a data source and connector manager with a graphical interface, practically identical to the *ODBC Data Source Administrator* on Windows. It operates under iODBC, an ODBC solution exclusive to macOS (although on macOS we can also use unixODBC, but we won't be able to take advantage of this specific utility).

Utilidades de *shell* We were able to run multiple Unix utilities in a *shell* for the development of the connector. Some important and lesser-known utilities that we employed include:

- **nm**: view the symbols contained in a library. This is useful for seeing which functions can be referenced in a compiled library. We used this to identify errors related to functions that other software was calling in our configuration library.
- **ipcs**: allows us to manually deal with shared memory files in RAM as those created with **shmget**. This was useful for debugging.

Conclusions and future work

4.1. Conclusions

We have finished our project having accomplished more than enough with all the objectives proposed in the section 1.2. So, starting from the ODBC Driver of MySQL, we have developed an ODBC Driver for DES for all the operating systems supported in DES: Windows, GNU/Linux and macOS, offering as well the possibility of choosing to work in ANSI or Unicode encoding (see section 3.5.7). Besides, we have been able to offer in addition an installer in every objective operating system, and a setup library not only in Windows but also in GNU/Linux (in this system, with the considerations we took in section 3.4).

However, we must acknowledge that despite our efforts, there is a not negligible number of bugs when dealing with the driver accross the wide variety of applications where it can be used. This is due to the high complexity of this software, with versions in different operating systems, and the variety of conventions that applications adopt when treating an ODBC Driver and sending them certain queries. We aim to allocate sufficient time in future works to solve them.

Figure 4.1 shows the list of the ODBC API functions and their state in the final product.

4.2. Future work

Throughout this work we have described a series of found problems and challenges whose resolutions exceed the limits of this work, either because of its complexity or the amount of time required. We list them as well as some new others in this section, and they are:

1. Implementation of dynamic cursors.
2. Implementation of unlimited size integers, in contrast with the current limited 64 bit integers.
3. Implementation of a setup library for the macOS version. Study how to achieve full compability between the GTK setup libraries in GNU/Linux and the

external GUI solution unixODBC refers to (`unixodbc-gui-qt`).

4. Though it is not the standard for a product of these characteristics, considering to support translations for the text messages throughout the suite beyond English.
5. Studying the viability of redesigning the architecture with advanced techniques that avoid the use of named pipes and named semaphores in certain platforms.
6. Thoroughly test the applications using ODBC to fix the bugs that the connector presents.

SQLAllocConnect (N/A)	(N/A)	SQLGetCursorName (M)	SQLRowCount (R)
SQLAllocEnv (N/A)	SQLDescribeCol (R)	SQLGetData (R)	SQLSetConnectAttr (M)
SQLAllocHandle (R)	SQLDescribeParam (M)	SQLGetDescField (M)	SQLSetConnectOption (N/A)
SQLAllocStmt (N/A)	SQLDisconnect (M)	SQLGetDescRec (R)	SQLSetCursorName (R)
SQLBindCol (R)	SQLDriverConnect (M)	SQLGetDiagField (M)	SQLSetDescField (M)
SQLBindParameter (R)	SQLDrivers (N/A)	SQLGetDiagRec (R)	SQLSetDescRec (R)
SQLBrowseConnect (X)	SQLEndTran (X)	SQLGetEnvAttr (M)	SQLSetEnvAttr (M)
SQLBulkOperations (M)	SQLError (N/A)	SQLGetFunctions (M)	SQLSetParam (N/A)
SQLCancel (X)	SQLExecDirect (M)	SQLGetInfo (M)	SQLSetPos (M)
SQLCancelHandle (X)	SQLExecute (M)	SQLGetStmtAttr (M)	SQLSetStmtAttr (M)
SQLCloseCursor (R)	SQLExtendedFetch (R)	SQLGetStmtOption (N/A)	SQLSetStmtOption (N/A)
SQLColAttribute (R)	SQLFetch (R)	SQLGetTypeInfo (M)	SQLSpecialColumns (M)
SQLColAttributes (N/A)	SQLFetchScroll (R)	SQLMoreResults (M)	SQLStatistics (M)
SQLColumnPrivileges (X)	SQLForeignKeys (M)	SQLNativeSql (R)	SQLTablePrivileges (X)
SQLColumns (M)	SQLFreeConnect (N/A)	SQLNumParams (R)	SQLTables (M)
SQLCompleteAsync (X)	SQLFreeEnv (N/A)	SQLNumResultCols (R)	SQLTransact (N/A)
SQLConnect (M)	SQLFreeHandle (R)	SQLParamData (X)	
SQLCopyDesc (R)	SQLFreeStmt (M)	SQLPrepare (R)	
SQLDataSources	SQLGetConnectAttr (M)	SQLPrimaryKeys (M)	
	SQLGetConnectOption (N/A)	SQLProcedureColumns (X)	
		SQLProcedures (X)	
		SQLPutData (X)	

N/A: not applicable (deprecated or a Driver Manager function).

M: modified from MyODBC for DESODBC.

R: reused completely from MyODBC.

X: not implemented due to DES' characteristics.

Figure 4.1: State of the ODBC API functions in DESODBC.

Bibliography

- [1] GEIGER, K. *Inside ODBC*. Microsoft Press, 1995.
- [2] ENGEL, D., GUYER, C., MASHA, T., RABELER, C., ROTH, J. and R., W. *ODBC programmer's reference - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/odbc-programmer-s-reference?view=sql-server-ver16> (last access: May 17, 2025).
- [3] SÁENZ-PÉREZ, F. *Datalog Educational System V1.1*, User's Manual. Technical Report 139-04, Facultad de Informática, Universidad Complutense de Madrid. 2024.
- [4] SÁENZ-PÉREZ, F. *Key Features*. Datalog Educational System. n.d. Avail. at https://www.fdi.ucm.es/profesor/fernand/des/html/key_features.html (last access: May 17, 2025).
- [5] SÁENZ-PÉREZ, F. *Datalog Educational System V6.7*, User's Manual. September 4, 2021. Avail. at <https://www.fdi.ucm.es/profesor/fernand/des/html/manual/manualDES.html> (last access: May 17, 2025).
- [6] SÁENZ-PÉREZ, F. *What's DES for?*. Datalog Educational System. n.d. Avail. at https://www.fdi.ucm.es/profesor/fernand/des/html/what_s_des_for_.html (last access: May 17, 2025).
- [7] SÁENZ-PÉREZ, F. *TFG. Propuestas de proyectos de Trabajo Fin de Grado (TFG) curso 2024-2025*. Fernando Sáenz Pérez. n.d. Avail. at <https://www.fdi.ucm.es/profesor/fernand/fsp/html/tfg.html> (last access: May 17, 2025).
- [8] BATCHELOR, D., CAI, S., COULTER, D., SATRAN, M., SHARKEY, K. and WHITE, S. *About dynamic-link libraries - Win32 apps*. Microsoft Learn. January 7, 2021. Avail. at <https://learn.microsoft.com/en-us/windows/win32/dlls/about-dynamic-link-libraries> (last access: May 17, 2025).
- [9] MySQL. *4.4 Building Connector/ODBC from a source distribution on Windows*. MySQL Connector/ODBC Developer Guide. n.d. Avail. at <https://dev.mysql.com/doc/connector-odbc/en/>

- `connector-odbc-installation-source-windows.html` (last access: May 17, 2025).
- [10] CAI, S., SATRAN, M. and WALKER, J. *Resource Compiler - Win32 apps*. Microsoft Learn. August 23, 2019. Avail. at <https://learn.microsoft.com/es-es/windows/win32/menurc/resource-compiler> (last access: May 17, 2025).
- [11] UNIXODBC. *unixODBC-Gui*. n.d. Avail. at <https://www.unixodbc.org/gui/> (last access: May 17, 2025).
- [12] SANDERS, R. E. *ODBC 3.5 Developer's Guide*. McGraw-Hill Professional, 1998.
- [13] ENGEL, D. AND GUYER, C. AND MASHA, T. AND RABELER, C. AND ROTH, J. AND WEST R. *Handles - ODBC API reference*. Microsoft Learn. October 18, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/develop-app/handles?view=sql-server-ver17> (last access: May 17, 2025).
- [14] ENGEL, D., GUYER, C., MASHA, T., RABELER, C., ROTH, J. and R., W. *Buffers - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/develop-app/buffers?view=sql-server-ver16> (last access: May 17, 2025).
- [15] ENGEL, D. AND GUYER, C. AND MASHA, T. AND RABELER, C. AND ROTH, J. AND WEST R. *Binding Parameters ODBC - ODBC API reference*. Microsoft Learn. October 18, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/develop-app/binding-parameters-odbc?view=sql-server-ver17> (last access: May 17, 2025).
- [16] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQL-GetInfo Function - ODBC API reference*. Microsoft Learn. October 18, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/sqlgetinfo-function?view=sql-server-ver16> (last access: May 17, 2025).
- [17] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLConnect Function - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/es-es/sql/odbc/reference/syntax/sqlconnect-function?view=sql-server-ver16> (last access: May 17, 2025).
- [18] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQL-DriverConnect Function - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/sqldriverconnect-function?view=sql-server-ver16> (last access: May 17, 2025).

- [19] ANDREWS, G. R. *Foundations of multithreaded, parallel, and distributed programming*. Wesley, University of Arizona, USA, 2000.
- [20] BATCHELOR, D., CAI, S., COULTER, D., JACOBS, M., SATRAN, M., SHARKEY, K. and WHITE, S. *Named Pipes - Win32 apps*. Microsoft Learn. July 1, 2021. Avail. at <https://learn.microsoft.com/en-us/windows/win32/ipc/named-pipes> (last access: May 17, 2025).
- [21] ANDERSSON, H., BATCHELOR, D., BRIDGE, K., CAI, S., COULTER, D., DOTERRER, K., KATSIOPIS, N., SATRAN, M. and SHARKEY, K. *Resource Compiler - Win32 apps*. Microsoft Learn. July 7, 2022. Avail. at <https://learn.microsoft.com/es-es/windows/win32/menurc/resource-compiler> (last access: May 17, 2025).
- [22] BATCHELOR, D., CAI, S., COULTER, D., SATRAN, M., SHARKEY, K., WHITE, S. ET AL. *Event Objects (Synchronization) - Win32 apps*. Microsoft Learn. January 30, 2021. Avail. at <https://learn.microsoft.com/en-us/windows/win32/sync/event-objects> (last access: May 17, 2025).
- [23] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLDisconnect Function - ODBC API reference*. Microsoft Learn. December 17, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/sqldisconnect-function?view=sql-server-ver16> (last access: May 17, 2025).
- [24] MICROSOFT. *CloseHandle Function - (handleapi.h)*. Microsoft Learn. October 13, 2021. Avail. at <https://learn.microsoft.com/es-es/windows/win32/api/handleapi/nf-handleapi-closehandle> (last access: May 17, 2025).
- [25] ENGEL, D., GUYER, C., MASHA, T., RABELER, C., ROTH, J. and R., W. *Driver Manager's Role in the Connection Process - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/develop-app/driver-manager-s-role-in-the-connection-process?view=sql-server-ver16> (last access: May 17, 2025).
- [26] UNIXODBC. *Closing a connection*. ODBC Programming Tutorial. n.d. Avail. at <https://www.unixodbc.org/doc/ProgrammerManual/Tutorial/close.html> (last access: May 17, 2025).
- [27] MICROSOFT. *OBJECT_ATTRIBUTES structure (ntdef.h) - Win32*. Microsoft Learn. August 9, 2023. Avail. at https://learn.microsoft.com/en-us/windows/win32/api/ntdef/ns-ntdef-_object_attributes (last access: May 17, 2025).
- [28] CAI, S. AND CASEY C. AND ROBERTSON, C. AND SHARKEY, K. AND WHITNEY T. ET AL. *atexit - C++, C, and Assembler*. Microsoft Learn. October 25, 2022. Avail. at <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/atexit?view=msvc-170> (last access: May 17, 2025).

-
- [29] ENGEL, D., GUYER, C., MASHA, T., RABELER, C., ROTH, J. and R., W. *Scrollable Cursor Types - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/develop-app/scrollable-cursor-types?view=sql-server-ver16> (last access: May 17, 2025).
- [30] ENGEL, D., GUYER, C., MASHA, T., RABELER, C., ROTH, J. and R., W. *Block cursors - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/develop-app/block-cursors?view=sql-server-ver16> (last access: May 17, 2025).
- [31] POTAPCHENKO, A., TSVIGUN, V., CISAR P., STARKEY J. ET AL. *FireBird ODBC/JDBC Driver 2.0 Manual*. August 30, 2020. Avail. at <https://www.firebirdsql.org/file/documentation/html/en/refdocs/fbodbc20/firebird-odbc-driver-20-manual.html> (last access: May 17, 2025).
- [32] ENGEL, D., GUYER, C., MASHA, T., RABELER, C., ROTH, J. and R., W. *Bookmarks (ODBC) - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/develop-app/bookmarks-odbc?view=sql-server-ver16> (last access: May 17, 2025).
- [33] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLSetPos Function - ODBC API reference*. Microsoft Learn. February 1, 2025. Avail. at <https://learn.microsoft.com/es-es/sql/odbc/reference/syntax/sqlsetpos-function?view=sql-server-ver17> (last access: May 17, 2025).
- [34] MySQL. *15.2.2 DELETE Statement*. MySQL 8.4 Reference Manual. n.d. Avail. at <https://dev.mysql.com/doc/refman/8.4/en/delete.html> (last access: May 17, 2025).
- [35] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLBulkOperations Function - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/sqlbulkoperations-function?view=sql-server-ver16> (last access: May 17, 2025).
- [36] SILBERSCHATZ, A., KORTH, H. F. and SUDARSHAN, S. *Database system concepts. Seventh edition*. McGraw-Hill Education, 2019.
- [37] MySQL. *8.1.3 Configuring Catalog and Schema Support*. MySQL Connector/ODBC Developer Guide. n.d. Avail. at <https://dev.mysql.com/doc/connector-odbc/en/connector-odbc-usagenotes-functionality-catalog-schema.html> (last access: May 17, 2025).

- [38] ENGEL, D., GUYER, C., MASHA, T., RABELER, C., ROTH, J. and R., W. *Pattern Value Arguments - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/develop-app/pattern-value-arguments?view=sql-server-ver16> (last access: May 17, 2025).
- [39] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLColumns Function - ODBC API reference*. Microsoft Learn. February 1, 2025. Avail. at <https://learn.microsoft.com/es-es/sql/odbc/reference/syntax/sqlcolumns-function?view=sql-server-ver16> (last access: May 17, 2025).
- [40] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLColumnPrivileges Function - ODBC API reference*. Microsoft Learn. February 1, 2025. Avail. at <https://learn.microsoft.com/es-es/sql/odbc/reference/syntax/sqlcolumnprivileges-function?view=sql-server-ver17> (last access: May 17, 2025).
- [41] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLSpecialColumns Function - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/sqlspecialcolumns-function?view=sql-server-ver16> (last access: May 17, 2025).
- [42] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLPrimaryKeys Function - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/sqlprimarykeys-function?view=sql-server-ver16> (last access: May 17, 2025).
- [43] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLForeignKeys Function - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/sqlforeignkeys-function?view=sql-server-ver16> (last access: May 17, 2025).
- [44] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLProcedures Function - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/sqlprocedures-function?view=sql-server-ver16> (last access: May 17, 2025).
- [45] MySQL. *15.1.17 CREATE PROCEDURE and CREATE FUNCTION Statements*. MySQL 8.4 Reference Manual. n.d. Avail. at <https://dev.mysql.com/doc/refman/8.4/en/create-procedure.html> (last access: May 17, 2025).
- [46] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLTables Function - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/es-es/sql/odbc/reference/>

- `syntax/sqltables-function?view=sql-server-ver16` (last access: May 17, 2025).
- [47] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *C Data Types - ODBC API reference*. Microsoft Learn. December 17, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/appendixes/c-data-types?view=sql-server-ver16> (last access: May 17, 2025).
- [48] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQL-GetData Function - ODBC API reference*. Microsoft Learn. December 17, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/sqlgetdata-function?view=sql-server-ver16> (last access: May 17, 2025).
- [49] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQL to C Data Conversion Examples - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/appendixes/sql-to-c-data-conversion-examples?view=sql-server-ver16> (last access: May 17, 2025).
- [50] CARLSSON M. ET AL. *SICStus Prolog User's Manual*. RISE Research Institutes of Sweden AB. Marzo de 2025. Avail. at <https://sicstus.sics.se/sicstus/docs/latest4/pdf/sicstus.pdf> (last access: May 17, 2025).
- [51] SWI-PROLOG. *SWI-Prolog Reference Manual*. n.d. Avail. at <https://www.swi-prolog.org/man/ari.html> (last access: May 17, 2025).
- [52] CAI, S., ENGEL, D., GUYER, C., MASHA, T., RABELER, C., ROTH, J. and R., W. *C Data Types in ODBC - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/develop-app/c-data-types-in-odbc?view=sql-server-ver16> (last access: May 17, 2025).
- [53] CAI, S., ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQL Data Types - ODBC API reference*. Microsoft Learn. 26 de agosto de 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/appendixes/sql-data-types?view=sql-server-ver16> (last access: May 17, 2025).
- [54] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQLStatistics Function - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/syntax/sqlstatistics-function?view=sql-server-ver16> (last access: May 17, 2025).
- [55] ENGEL, D., GUYER, C., RABELER, C., ROTH, J., WEST R. ET AL. *SQL-GetDiagRec Function - ODBC API reference*. Microsoft Learn. June 25, 2024. Avail. at <https://learn.microsoft.com/en-us/sql/odbc/reference/>

- `syntax/sqlgetdiagrec-function?view=sql-server-ver16` (last access: May 17, 2025).
- [56] COMPART AG. *Unicode Character U+24B62*. 2021. Avail. at <https://www.compart.com/en/unicode/U+24B62> (last access: May 17, 2025).
- [57] MySQL. *MySQL Connector/C++ 9.2 Developer Guide*. n.d. Avail. at <https://dev.mysql.com/doc/connector-cpp/9.2/en/> (last access: May 17, 2025).

